



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temario para la preparación de la Oposición al Cuerpo Superior de
Sistemas y Tecnologías de la Información de la Administración del
Estado.

TEMAS ESPECÍFICOS – BLOQUE B

III. Ingeniería de los sistemas de información

**Tema 99. La estimación de recursos y esfuerzo en el desarrollo de
sistemas de información**

AUTOR: Miguel Ángel Rodrigo Muñoz.

REVISOR: Alfonso Sancho Miró.

Creación: septiembre 2022

ÍNDICE

1	EL PAPEL DE LOS ALGORITMOS EN LA COMPUTACIÓN.	4
1.1.1	INTRODUCCIÓN.....	4
1.1.2	ALGORITMOS	4
1.1.3	¿QUÉ TIPO DE PROBLEMAS SE RESUELVEN MEDIANTE ALGORITMOS?.....	4
1.1.3.1	TÉCNICA.....	6
1.1.3.2	PROBLEMAS DIFÍCILES.....	7
1.1.3.3	PARALELISMO.....	7
1.1.4	ALGORITMOS COMO TECNOLOGÍA.....	8
1.1.4.1	EFICIENCIA.....	8
1.1.5	ALGORITMOS Y OTRAS TECNOLOGÍAS	9
1.1.6	ANÁLISIS DE ALGORITMOS.....	10
1.1.7	ORDEN DE INSERCIÓN.....	10
1.1.8	INVARIANTES DE BUCLE Y CORRECCIÓN DE LA ORDENACIÓN POR INSERCIÓN	12
1.1.9	ANÁLISIS DE ALGORITMOS	13
1.1.9.1	ANÁLISIS DE ORDENACIÓN POR INSERCIÓN	14
1.1.9.2	ANÁLISIS DEL PEOR CASO Y CASO PROMEDIO	16
1.1.10	ORDEN DE CRECIMIENTO.....	17
1.1.11	DISEÑO DE ALGORITMOS.....	17
1.1.11.1	EL ENFOQUE DE DIVIDE Y VENCERÁS	17
1.1.11.2	ANÁLISIS DE ALGORITMOS DE DIVIDE Y VENCERÁS	23
1.1.11.3	ANÁLISIS DE MERGESORT.....	24
1.1.11.4	MERGESORT: ANÁLISIS EMPÍRICO	27
1.1.11.5	DIVIDE Y VENCERÁS: PRUEBA POR IMAGEN.....	28
1.2	TÉCNICAS PARA MEDIR Y ESTIMAR EL TAMAÑO DEL SOFTWARE A PARTIR DE LA COMPLEJIDAD DE SUS FUNCIONALIDADES.....	29
1.2.1	INTRODUCCIÓN.....	29
1.2.2	PASOS PARA ELABORAR ESTIMACIONES DE PROYECTOS DE SOFTWARE	29
1.2.3	EJEMPLO PARA MEDIR EL SOFTWARE O COMPONENTE A DESARROLLAR	32
1.2.4	MEDICIÓN DE ATRIBUTOS INTERNOS DEL PRODUCTO. LONGITUD	35
1.2.4.1	MÉTRICAS DE HALSTEAD [HALSTEAD, 1977]	36
1.2.4.3	MEDICIÓN DE ATRIBUTOS INTERNOS DEL PRODUCTO. FUNCIONALIDAD	37
1.2.4.4	MODIFICACIONES Y ALTERNATIVAS A FPA	38
1.2.4.5	MEDICIÓN DE ATRIBUTOS INTERNOS DEL PRODUCTO	39
1.2.5	COSMIC-FFP [ISO/IEC 19761:2003]	39
1.2.6	UCP (USE CASE POINTS).....	39
1.2.7	ENFOQUE COCOMO	41
1.2.8	MÉTRICAS BANG	41
1.2.9	COMPLEJIDAD.....	42
1.3	COMPLEJIDAD ALGORÍTMICA	44

1.3.1	EFICIENCIA Y COMPLEJIDAD	44
1.3.2	REGLAS GENERALES PARA EL CÁLCULO DEL NÚMERO DE OE	46
1.3.3	COTAS DE COMPLEJIDAD. MEDIDAS ASINTÓTICAS	47
1.3.3.1	COTA SUPERIOR. NOTACIÓN O	48
1.3.3.2	PROPIEDADES DE O	49
1.3.3.3	COTA INFERIOR. NOTACIÓN Ω	49
1.3.3.4	PROPIEDADES DE Ω	50
1.3.3.5	ORDEN EXACTO. NOTACIÓN Θ	50
1.3.3.6	OBSERVACIONES SOBRE LAS COTAS ASINTÓTICAS	52
1.3.4	RESOLUCIÓN DE ECUACIONES EN RECURRENCIA	53
1.3.4.1	RECURRENCIAS HOMOGÉNEAS	53
1.3.4.2	RECURRENCIAS NO HOMOGÉNEAS	54
2	RESUMEN ESQUEMÁTICO	0
3	GLOSARIO	2
4	TEST	4
4.1	PREGUNTAS DE TEST	4
4.2	SOLUCIONES A LAS PREGUNTAS DE TEST	6
5	BIBLIOGRAFÍA BÁSICA	7
5.1	BIBLIOGRAFÍA BÁSICA	7
5.2	BIBLIOGRAFÍA PARA AMPLIAR EL TEMA	8

1 El papel de los Algoritmos en la computación.

1.1.1 Introducción

Esta parte hará comenzar a pensar en diseñar y analizar algoritmos. Tiene la intención de ser una introducción sobre cómo especificar algoritmos y algunas de las estrategias de diseño que usaremos y muchas de las ideas fundamentales que se usan en el análisis de algoritmos. Este capítulo proporciona una descripción general de los algoritmos y su lugar en los sistemas informáticos modernos. Este capítulo define qué es un algoritmo y enumera algunos ejemplos. También argumenta que deberíamos considerar los algoritmos como una tecnología, junto con tecnologías como el hardware, interfaces gráficas de usuario, sistemas orientados a objetos y redes.

1.1.2 Algoritmos

De manera informal, un algoritmo es cualquier procedimiento computacional bien definido que toma algún valor, o conjunto de valores, como entrada y produce algún valor, o conjunto de valores, como salida. Por tanto, un algoritmo es una secuencia de pasos computacionales que transforman la entrada en la salida. También podemos ver un algoritmo como una herramienta para resolver un problema computacional bien definido. El enunciado del problema específica, en términos generales, la relación entrada / producto deseado. El algoritmo describe un procedimiento computacional específico para lograr esa relación entrada / salida. Por ejemplo, es posible que necesitemos ordenar una secuencia de números en orden no decreciente. Este problema surge con frecuencia en la práctica y proporciona un terreno fértil para introducir muchas técnicas de diseño estándar y herramientas de análisis. Así es como definimos formalmente el problema de clasificación:

Input: Una secuencia de números $(a_1, a_2, a_3, \dots, a_N)$

Output: Una permutación (reordering) $(a_1', a_2', a_3', \dots, a_N')$ de la secuencia de entrada, tal que: $(a_1' \leq a_2' \leq a_3' \leq \dots \leq a_N')$

Debido a que muchos programas lo utilizan como un paso intermedio, la clasificación es una operación fundamental en la informática. Como resultado, tenemos una gran cantidad de buenos algoritmos de clasificación a nuestra disposición. Qué algoritmo es mejor para una aplicación determinada depende, entre otros factores, de la cantidad de elementos que se van a clasificar, la medida en que los elementos ya están algo ordenados, las posibles restricciones en los valores de los elementos, la arquitectura del ordenador y el tipo de dispositivos de almacenamiento utilizar: memoria principal, discos o incluso cintas.

Se dice que un algoritmo es correcto si, para cada instancia de entrada, se detiene con la salida correcta. Decimos que un algoritmo es correcto, si resuelve el problema computacional dado. Un algoritmo incorrecto podría no detenerse en absoluto en algunas instancias de entrada, o podría detenerse con una respuesta incorrecta. Al contrario de lo que cabría esperar, los algoritmos incorrectos a veces pueden ser útiles, si podemos controlar su tasa de error.

Por lo general, sin embargo, sólo nos ocuparemos de los algoritmos correctos. Un algoritmo se puede especificar en texto, como un programa de ordenador o incluso como un diseño de hardware. El único requisito es que la especificación debe proporcionar una descripción precisa del procedimiento de cálculo a seguir.

1.1.3 ¿Qué tipo de problemas se resuelven mediante algoritmos?

La ordenación no es de ninguna forma el único problema computacional para el que se han desarrollado algoritmos.

Las aplicaciones prácticas de los algoritmos son omnipresentes e incluyen los siguientes ejemplos:

El Proyecto Genoma Humano ha logrado grandes avances hacia los objetivos de identificar los 100.000 genes del ADN humano, determinando las secuencias de los 3000 millones de pares de bases químicas que componen el ADN humano, almacenar esta información en bases de datos y desarrollar herramientas para el análisis de datos. Cada uno de estos pasos requiere sofisticados algoritmos.

Muchos métodos para resolver estos problemas biológicos utilizan ideas que permiten a los científicos realizar tareas utilizando los recursos de manera eficiente. Los ahorros son en tiempo, tanto humanos como mecánicos, y monetarios.

Internet permite que personas de todo el mundo accedan y recuperen rápidamente grandes cantidades de información. Con la ayuda de algoritmos inteligentes, los nodos de Internet pueden gestionar y manipular este gran volumen de datos. Ejemplos de problemas que hacen un uso esencial de los algoritmos incluyen encontrar rutas óptimas por las que viajarán los datos o usar un motor de búsqueda para encontrar de forma rápida, páginas en las que reside una información particular. El comercio electrónico permite que los bienes y servicios se negocien e intercambien electrónicamente, y depende de la privacidad de la información personal, como números de tarjetas de crédito, contraseñas y extractos bancarios. Las tecnologías centrales utilizadas en el comercio electrónico incluyen la criptografía de clave pública y las firmas, que se basan en algoritmos numéricos y la teoría de números. Las empresas manufactureras y otras empresas comerciales a menudo necesitan asignar recursos escasos de forma óptima. Una compañía petrolera puede querer saber dónde colocar sus pozos para maximizar su beneficio. Un candidato político puede querer determinar dónde gastar dinero contratando publicidad de campaña para maximizar las posibilidades de ganar una elección. Es posible que una aerolínea desee asignar tripulaciones a los vuelos de la manera más eficiente y barata posible, asegurándose de que cada vuelo esté cubierto y de que se cumplan las regulaciones gubernamentales con respecto a la programación de la tripulación. Es posible que un proveedor de servicios de Internet desee determinar dónde colocar recursos adicionales para atender a sus clientes de manera más eficaz.

Todos estos son ejemplos de problemas que pueden resolverse usando programación lineal. Se nos dan dos secuencias ordenadas de símbolos, $X = \{X_1, X_2, X_3, \dots, X_N\}$ e $Y = \{Y_1, Y_2, Y_3, \dots, Y_N\}$ y deseamos encontrar la subsecuencia común más larga de X e Y . Una subsecuencia de X es solo X con algunos (o posiblemente todos o ninguno) de sus elementos eliminados. Por ejemplo, una subsecuencia de (A, B, C, D, E, F, G) sería (B, C, E, G) ; La longitud de la subsecuencia común más larga de X e Y da una medida de cuán similares son estas dos secuencias. Por ejemplo, si las dos secuencias son pares de bases en cadenas de ADN, entonces podríamos considerarlas similares si tienen una subsecuencia común larga. Si X tiene m símbolos e Y tiene n símbolos, entonces X e Y tienen 2^m y 2^n subsecuencias posibles, respectivamente.

Seleccionar todas las subsecuencias posibles de X e Y , y hacerlas coincidir podría llevar un tiempo prohibitivamente largo a menos que m y n sean muy pequeños. Se nos da un diseño mecánico en términos de una biblioteca de partes, donde cada parte puede incluir instancias de otras partes, y necesitamos listar las partes en orden para que cada parte aparezca antes que cualquier parte que la use. Si el diseño comprende n partes, entonces hay $n!$ órdenes posibles.

Debido a que la función factorial crece más rápido que incluso una función exponencial, no podemos generar de manera factible cada orden posible y luego verificar que, dentro de ese orden, cada parte aparece antes que las partes que la usan (a menos que tengamos solo unas pocas partes).

Otro ejemplo: Se nos dan n puntos en el plano y deseamos encontrar el casco convexo de estos puntos. El casco convexo es el polígono convexo más pequeño que rodea todos los puntos. Intuitivamente, podemos pensar que cada punto está representado por un clavo que sobresale de una tabla. El casco convexo estaría representado por una banda de goma

ajustada, que rodea todos los clavos. Cada clavo alrededor del cual gira la goma elástica es un vértice del casco convexo.

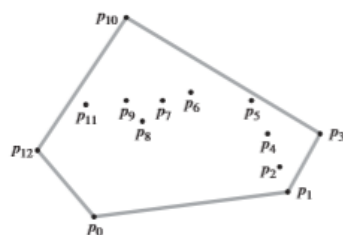


Figura 1.1. Casco convexo de una serie de puntos del plano.

Cualquiera de los 2^n subconjuntos de puntos podrían ser los vértices del casco convexo. Saber qué puntos son vértices del casco convexo tampoco es suficiente, ya que también necesitamos saber el orden en que aparecen. Por lo tanto, hay muchas opciones para los vértices del casco convexo.

Esta lista de ejemplos, está lejos de ser exhaustiva, pero exhiben dos características que son comunes a muchos problemas algorítmicos interesantes:

1. Tienen muchas soluciones candidatas, la inmensa mayoría de las cuales no resuelven el problema en cuestión. Encontrar una que lo haga, o una que sea "mejor", puede presentar un gran reto.
2. Tienen aplicaciones prácticas. De los problemas de la lista anterior, encontrar el camino más corto proporciona los ejemplos más fáciles. Una empresa de transporte, como una empresa de camiones o ferrocarriles, tiene un interés financiero en encontrar los caminos más cortos a través de una red de carreteras o ferrocarriles porque tomar caminos más cortos resulta en menores costos de trabajo y combustible. O un nodo de enrutamiento en Internet puede necesitar encontrar la ruta más corta a través de la red para enrutar un mensaje rápidamente.
3. No todos los problemas resueltos por algoritmos tienen un conjunto de soluciones candidatas fácilmente identificables. Por ejemplo, suponga que se nos da un conjunto de valores numéricos que representan muestras de una señal y queremos calcular la transformada discreta de Fourier de estas muestras. La transformada discreta de Fourier convierte el dominio del tiempo en el dominio de la frecuencia, produciendo un conjunto de coeficientes numéricos, por lo que podemos determinar la presencia de varias frecuencias en la señal muestreada. Además de estar en el corazón del procesamiento de señales, las transformadas discretas de Fourier tienen aplicaciones en la compresión de datos y la multiplicación de grandes polinomios y de números enteros.

1.1.3.1 Técnica

. El centro de nuestro estudio trata sobre algoritmos eficientes. Nuestra medida habitual de eficiencia es la velocidad, es decir, cuánto tarda un algoritmo en producir su resultado. Sin embargo, existen algunos problemas para los que no se conoce una solución eficaz.

1.1.3.2 Problemas difíciles.

Un subconjunto interesante de estos problemas se conoce como NP-completo. El interés de los problemas NP-completos radica en:

- Primero, aunque nunca se ha encontrado un algoritmo eficiente para un problema NP-completo, nadie lo ha probado nunca. En otras palabras, nadie sabe si existen o no algoritmos eficientes para problemas NP-completos.
- En segundo lugar, el conjunto de problemas NP-completos tiene la propiedad notable de que, si un algoritmo eficiente existe para cualquiera de ellos, existen algoritmos eficientes para todos ellos. Esta relación entre los problemas NP-completos hace que la falta de soluciones eficientes sea aún más tentadora.
- En tercer lugar, varios problemas NP-completos son similares, pero no idénticos, a los problemas para los que conocemos algoritmos eficientes. Los científicos están intrigados por cómo un pequeño cambio en el planteamiento del problema puede provocar un gran cambio en la eficiencia del algoritmo más conocido.

Se debe conocer los problemas NP-completos porque algunos de ellos surgen sorprendentemente a menudo en aplicaciones reales. Si se le pide que produzca un algoritmo eficiente para un problema NP-completo, es probable que dedique mucho tiempo a una búsqueda infructuosa. Si puede demostrar que el problema es NP-completo, puede dedicar su tiempo a desarrollar un algoritmo eficiente que proporcione una buena solución, pero no la mejor posible.

Como ejemplo concreto, considere una empresa de entrega con un depósito central. Cada día, carga cada camión de reparto en el depósito y lo envía para entregar mercancías a varias direcciones. Al final del día, cada camión debe retroceder al depósito para que esté listo para ser cargado para el día siguiente. Para reducir los costos, la empresa desea seleccionar un orden de entrega que produzca la distancia total más baja recorrida por cada camión. Este problema es el conocido "problema del visitador comercial" y es NP-completo. No tiene un algoritmo eficiente conocido. Sin embargo, bajo ciertos supuestos, conocemos algoritmos eficientes que dan una distancia total que no está muy por encima de la más corta posible. Son los llamados "algoritmos de aproximación".

1.1.3.3 Paralelismo

Durante muchos años, pudimos contar con un aumento constante de la velocidad del reloj del procesador. Sin embargo, las limitaciones físicas presentan un obstáculo fundamental para las velocidades de reloj cada vez mayores: debido a que la densidad de potencia aumenta super linealmente con la velocidad del reloj, los chips corren el riesgo de derretirse una vez que sus velocidades de reloj se vuelven lo suficientemente altas.

Por tanto, para realizar más cálculos por segundo, los chips se diseñan para contener no solo uno, sino varios "núcleos" de procesamiento. Podemos comparar estos ordenadores multinúcleo con varios procesadores secuenciales en un solo chip; en otras palabras, son un tipo de "computación paralela". Para obtener el mejor rendimiento de los ordenadores multinúcleo, necesitamos diseñar algoritmos teniendo en cuenta el paralelismo. Este modelo tiene ventajas desde un punto de vista teórico y constituye la base de varios programas informáticos exitosos, incluido un programa de campeonato de ajedrez.

1.1.4 Algoritmos como tecnología

Supongamos que los ordenadores fueran infinitamente rápidos y la memoria estuviera libre. ¿habría alguna razón para estudiar algoritmos? La respuesta es sí, aunque sólo sea por eso, sería bueno demostrar que su método de solución termina y lo hace con la respuesta correcta. Si los ordenadores fueran infinitamente rápidos, cualquier método correcto para resolver un problema sería suficiente. Probablemente se desee que la implementación esté dentro de los límites de las buenas prácticas de ingeniería de software (por ejemplo, la implementación debe estar bien diseñada y documentada), pero normalmente se usaría el método más fácil de implementar. Por supuesto, los ordenadores pueden ser rápidos, pero no infinitamente rápidos. Y la memoria puede ser barata, pero no gratuita. Por tanto, el tiempo es un recurso limitado, al igual que el espacio en la memoria. Se deben utilizar estos recursos con prudencia, y los algoritmos que sean eficientes en términos de tiempo o espacio ayudarán a hacerlo.

1.1.4.1 Eficiencia

Los diferentes algoritmos ideados para resolver el mismo problema a menudo difieren dramáticamente en su eficiencia. Estas diferencias pueden ser mucho más significativas que las debidas al hardware y al software. Como ejemplo, veremos dos algoritmos para ordenar. El primero, conocido como “ordenación por inserción”, toma un tiempo aproximadamente igual a $c_1 n^2$ para ordenar n elementos, donde c_1 es una constante que no depende de n . Es decir, lleva un tiempo aproximadamente proporcional a n^2 . El segundo, el tipo de combinación, toma un tiempo aproximadamente igual a $c_2 n \log n$, donde $(\log n)$ significa $\log_2(n)$ y c_2 es otra constante que tampoco depende de n . La “ordenación por inserción” normalmente tiene un factor constante más pequeño que la ordenación por fusión o combinación (**mergesort**), de modo que $c_1 < c_2$. Veremos que los factores constantes pueden tener un impacto mucho menor en el tiempo de ejecución que la dependencia del tamaño de entrada n .

Vamos a representar, como veremos posteriormente, el tiempo de ejecución de la ordenación por inserción como $c_1 n^2$ y el tiempo de ejecución de la ordenación **mergesort** como $c_2 n \log n$. Luego vemos que donde la ordenación por inserción tiene un factor de n en su tiempo de ejecución, la ordenación por combinación tiene un factor de $\log n$, que es mucho más pequeño. (Por ejemplo, cuando $n = 1000$, $\log n$ es aproximadamente 10 , y cuando n es igual a un **millón**, $\log n$ es aproximadamente solo 20 .)

Aunque la ordenación por inserción generalmente se ejecuta más rápido que la ordenación por mergesort, para tamaños de entrada pequeños, una vez que el tamaño de entrada n se vuelve lo suficientemente grande, la ventaja del tipo de **mergesort** de $\log n$ frente a n compensará con creces la diferencia en los factores constantes. No importa cuánto más pequeño sea c_1 que c_2 , siempre habrá un punto de cruce más allá del cual la ordenación por **mergesort** es más rápida.

Para un ejemplo concreto, comparamos un ordenador muy rápido (ordenador A) que ejecuta la ordenación por inserción contra un ordenador más lento (ordenador B) que ejecuta la ordenación por **mergesort**. Cada uno de ellos debe ordenar una matriz de 10 millones de números. (Aunque 10 millones de números puede parecer mucho, si los números son enteros de ocho bytes, ocupan en la entrada alrededor de 80 megabytes).

Suponga que el ordenador **A** ejecuta **10 mil millones** de instrucciones por segundo y el ordenador **B** ejecuta solo **10 millones** de instrucciones por segundo, por lo que el ordenador A es 1000 veces más rápido que el ordenador B en potencia de cómputo bruta. Para hacer la diferencia aún más dramática, suponga que el programador más cualificado del mundo ordena la inserción de códigos en lenguaje de máquina para el ordenador A, y que el código resultante requiere $2n^2$ instrucciones para ordenar n números.

Suponga además que solo un programador promedio implementa **mergesort**, usando un lenguaje de alto nivel con un compilador ineficiente, con el código resultante tomando **$50n \log n$** instrucciones. Para ordenar 10 millones de números, el ordenador **A** toma más de 5,5 horas, mientras que el ordenador **B** toma: 1163 segundos (menos de 20 minutos):

Al usar un algoritmo cuyo tiempo de ejecución crece más lentamente, incluso con un compilador deficiente, el ordenador B es más de 17 veces más rápido que el ordenador A. La ventaja de la ordenación por **mergesort** es aún más pronunciada cuando ordenamos 100 millones de números: donde la ordenación por inserción demora más de 23 días, la ordenación por **mergesort** toma menos de cuatro horas. En general, a medida que aumenta el tamaño del problema, también lo hace la ventaja relativa de la ordenación por mergesort.

1.1.5 Algoritmos y otras tecnologías

El ejemplo anterior muestra que deberíamos considerar los algoritmos, como el hardware, es decir, como una tecnología. El rendimiento total del sistema depende tanto de elegir algoritmos eficientes como de elegir hardware rápido. Así como se están logrando rápidos avances en otras tecnologías informáticas, también se están realizando en algoritmos. Quizás se cuestione si los algoritmos son realmente tan importantes en los ordenadores contemporáneos a la luz de otras tecnologías avanzadas, tales como: arquitecturas informáticas avanzadas, tecnologías de fabricación, interfaces gráficas de usuario (GUI intuitivas y fáciles de usar), sistemas orientados a objetos, tecnologías web integradas y redes rápidas, tanto cableadas como inalámbricas. La respuesta es: sí.

Aunque algunas aplicaciones no requieren explícitamente contenido algorítmico a nivel de la aplicación (como las aplicaciones simples basadas en la Web), muchas sí lo hacen. Por ejemplo, considere un servicio basado en la Web que determina cómo viajar de un lugar a otro. Su implementación se basaría en hardware rápido, una interfaz gráfica de usuario, redes WAN y también posiblemente uso de orientación a objetos en el lenguaje de desarrollo. Sin embargo, también requeriría algoritmos para ciertas operaciones, como encontrar rutas (probablemente usando un algoritmo de cálculo de ruta más corta), representar mapas y buscar direcciones.

Además, incluso una aplicación que no requiere contenido algorítmico a nivel de aplicación depende en gran medida de los algoritmos. ¿La aplicación se basa en hardware rápido? El diseño de hardware utilizó algoritmos. ¿La aplicación se basa en interfaces gráficas de usuario? El diseño de cualquier GUI se basa en algoritmos. ¿La aplicación depende de la red? El enrutamiento en redes se basa en gran medida en algoritmos. ¿La aplicación estaba escrita en un lenguaje diferente al código máquina? Luego fue procesado por un compilador, intérprete o ensamblador, todos los cuales hacen un uso extensivo de algoritmos.

Los algoritmos son el núcleo de la mayoría de las tecnologías utilizadas en los ordenadores contemporáneos. Además, con las capacidades cada vez mayores de éstos, las usamos para resolver problemas más grandes que nunca. Como vimos en la comparación anterior entre la "ordenación por inserción" y la ordenación por **mergesort**, es en los tamaños de problemas más grandes donde las diferencias de eficiencia entre los algoritmos se vuelven particularmente prominentes. Tener una base sólida de conocimientos y técnicas algorítmicas es una característica que distingue a los programadores verdaderamente hábiles de los novatos. Con la tecnología informática moderna, se pueden realizar algunas tareas sin saber mucho sobre algoritmos, pero con una buena experiencia en algoritmos, se puede hacer mucho más.

1.1.6 Análisis de Algoritmos.

Comenzamos examinando el algoritmo de “ordenación por inserción” para ordenar elementos. Definimos un “pseudocódigo” que debería ser familiar si ha realizado programación anteriormente, y se usa para mostrar cómo especificaremos los algoritmos.

Habiendo especificado el algoritmo de ordenación por inserción, argumentaremos que ordena correctamente y analizaremos su tiempo de ejecución. El análisis introduce una notación que se centra en cómo aumenta ese tiempo con la cantidad de elementos que se van a clasificar. Después de la discusión sobre la ordenación por inserción, se presentará el enfoque de “divide y vencerás” en el diseño de algoritmos y se usará para desarrollar un algoritmo llamado ordenación “mergesort”. Terminamos con un análisis del tiempo de ejecución de este tipo de algoritmo.

1.1.7 Orden de inserción

Nuestro primer algoritmo, la ordenación por inserción, resuelve el problema de ordenación:

Entrada: Una secuencia de n números ($a^1, a^2, a^3, \dots, a^n$).

Salida: Una permutación (reordenamiento) ($a'^1, a'^2, a'^3, \dots, a'^n$) de la secuencia de entrada tal que

$$a'^1 \leq a'^2 \leq \dots \leq a'^n.$$

Los números que se desea ordenar también se conocen como claves. Aunque conceptualmente estamos ordenando una secuencia, la entrada llega en forma de una matriz con n elementos. En este capítulo, normalmente se describen los algoritmos como programas escritos en un pseudocódigo que es similar en muchos aspectos a C, C++, Java, Python o Pascal. Si ha sido introducido a cualquiera de estos lenguajes, no debería tener problemas leer los algoritmos presentados.



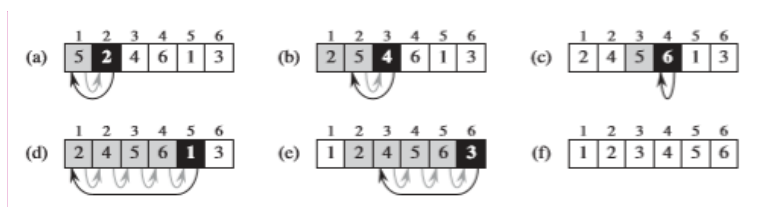
Figura 1.2 Clasificar una mano de cartas usando la clasificación por inserción.

Lo que separa el pseudocódigo del código “real” es que, en el pseudocódigo, se emplea cualquier método expresivo que sea más claro y conciso para especificar un algoritmo dado. Otra diferencia entre el pseudocódigo y el código real es que el pseudocódigo no suele preocuparse por cuestiones de ingeniería de software. Los problemas de abstracción de datos, modularidad y manejo de errores a menudo se ignoran para transmitir la esencia del

algoritmo de manera más concisa. Se comienza con la “ordenación por inserción”, que es un algoritmo eficiente para ordenar una pequeña cantidad de elementos.

La “clasificación por inserción” funciona de la misma forma que muchas personas clasifican una mano de naipes. Comenzamos con la mano izquierda vacía y las cartas boca abajo sobre la mesa. Luego retiramos una tarjeta a la vez de la mesa y la insertamos en la posición correcta en la mano izquierda. Para encontrar la posición correcta de una carta, la comparamos con cada una de las cartas que ya están en la mano, de derecha a izquierda, como se ilustra en la Figura 1.2. En todo momento, las cartas que se encuentran en la mano izquierda están ordenadas, y estas cartas eran originalmente las cartas superiores de la pila de la mesa.

Se presenta el pseudocódigo para ordenación por inserción como un procedimiento llamado INSERTIONSORT, que toma como parámetro una matriz $A[1,2,3...N]$ que contiene una secuencia de longitud n que se va a ordenar. (En el código, el número n de elementos en A se denota por $A.length$). El algoritmo ordena los números de entrada en su lugar: reordena los números dentro de la matriz A , con como máximo un número constante de ellos almacenados fuera de la matriz en cualquier momento. La matriz de entrada A contiene la secuencia de salida ordenada cuando finaliza el procedimiento INSERTION-SORT.



Comentado [MÁRM1]:

Figura 1.3. El funcionamiento de INSERTION-SORT en la matriz $A = \{5, 2, 4, 6, 1, 3\}$. Los índices de matriz aparecen encima de los rectángulos y los valores almacenados en las posiciones de la matriz aparecen dentro de los rectángulos. (a) - (e) Las iteraciones del bucle **for** de las líneas 1–8. En cada iteración, el rectángulo negro contiene la clave tomada de $A[j]$, que se compara con los valores de los rectángulos sombreados a su izquierda en la prueba de la línea 5. Las flechas sombreadas muestran los valores de la matriz movidos una posición a la derecha en la línea 6, y negro las flechas indican a dónde se mueve la clave en la línea 8. (f) La matriz ordenada final.

INSERTION-SORT.A

```

1  for  $j = 2$  to  $A.length$ 
2       $key = A[j]$ 
3      // Insert  $A[j]$  into the sorted sequence  $A[1..j-1]$ 
4       $i = j-1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i+1] = A[i]$ 
7           $i = i-1$ 
8       $A[i+1] = key$ 

```

1.1.8 Invariantes de bucle y corrección de la ordenación por inserción

La figura 1.3 muestra cómo funciona este algoritmo para $A = \{5, 2, 4, 6, 1, 3\}$. El índice j indica la "carta actual" que se inserta en la mano. Al comienzo de cada iteración del ciclo 'for', que está indexado por j , la submatriz que consta de los elementos $A[1...j-1]$ constituye la mano actualmente ordenada, y la submatriz restante $A[j...n]$ corresponde al montón de cartas que aún están en la mesa. De hecho, los elementos $A[1...j-1]$ son los elementos originalmente en las posiciones 1 a $(j-1)$, pero ahora ordenados. Declaramos estas propiedades de $A[1...j-1]$ formalmente como un **ciclo invariante**:

Al comienzo de cada iteración del bucle 'for' de las líneas 1-8, la **submatriz** $A[1...j-1]$ consta de los elementos originalmente en $A[1...j-1]$, pero ordenados.

Usamos invariantes de bucle para ayudarnos a comprender por qué un algoritmo es correcto. Debemos mostrar tres cosas sobre un ciclo invariante:

Inicialización: es cierto antes de la primera iteración del bucle.

Mantenimiento: si es verdadero antes de una iteración del bucle, permanece verdadero antes de la siguiente iteración.

Terminación: cuando el bucle termina, la invariante nos da una propiedad útil que ayuda a demostrar que el algoritmo es correcto.

Obsérvese la similitud con la inducción matemática, donde para probar que se cumple una propiedad, se prueba un caso base y se realiza un paso inductivo. Aquí, mostrar que el invariante se cumple antes de la primera iteración corresponde al caso base, y mostrar que el invariante se cumple de una iteración a otra corresponde al paso inductivo.

La tercera propiedad es quizás la más importante, ya que usamos el invariante de bucle para mostrar la corrección. Por lo general, usamos el invariante de bucle junto con la condición de que provocó la terminación del bucle. La propiedad de terminación difiere de cómo usamos habitualmente la inducción matemática, en la que aplicamos el paso inductivo infinitamente; aquí, detenemos la "inducción" cuando termina el bucle.

Veamos cómo se mantienen estas propiedades para la ordenación por inserción.

Inicialización: Se comienza mostrando que el invariante del bucle se mantiene antes de la primera iteración del ciclo, cuando $j = 2$.¹ La submatriz $A[1...j-1]$, por lo tanto, consiste solo en el elemento $A[1]$, que de hecho es el elemento original en $A[1]$. Además, esta submatriz está ordenada (trivialmente, por supuesto), lo que muestra que el invariante del bucle se mantiene antes de la primera iteración del ciclo.

Mantenimiento: A continuación, abordamos la segunda propiedad: mostrar que cada iteración mantiene el ciclo invariante. De manera informal, el cuerpo del bucle 'for' funciona moviendo $A[j-1]$, $A[j-2]$, $A[j-3]$, y así sucesivamente una posición hacia la derecha hasta encontrar la posición adecuada para $A[j]$ (líneas 4-7), en cuyo punto inserta el valor de $A[j]$ (línea 8). La submatriz $A[1...j]$ entonces consta de los elementos originalmente en $A[1...j]$, pero ordenados. Incrementar j para la siguiente iteración del ciclo 'for' conserva el bucle invariante.

Un tratamiento más formal de la segunda propiedad nos requeriría enunciar y mostrar un ciclo invariante para el bucle 'while' de las líneas 5-7. En este punto, sin embargo, es preferible no empantanarnos en tal formalismo, por lo que confiamos en nuestro análisis informal para mostrar que la segunda propiedad es válida para el bucle externo.

¹ Cuando el ciclo es un ciclo **for**, el momento en el que verificamos el invariante del ciclo justo antes de la primera iteración es inmediatamente después de la asignación inicial a la variable contador de ciclo y justo antes de la primera prueba en el encabezado del ciclo. En el caso de INSERTION-SORT, este tiempo es después de asignar 2 a la variable j pero antes de la primera comprobación de si $j \leq A.length$.

Finalización: Finalmente, examinamos qué sucede cuando termina el bucle. La condición que hace que el bucle 'for' termine es que $j > A.length = n$. Debido a que cada iteración del bucle aumenta j en 1, debemos tener $j = n + 1$ en ese momento. Sustituyendo $(n + 1)$ por j en la redacción del bucle invariante, tenemos que la submatriz $A[1...n]$ consta de los elementos originalmente en $A[1....n]$, pero ordenados. Observando que la submatriz $A[1....n]$ es la submatriz completa, llegamos a la conclusión de que toda la matriz está ordenada. Por tanto, el algoritmo es correcto. Usaremos este método de invariantes de bucle para mostrar la corrección de más algoritmos.

1.1.9 Análisis de algoritmos

Analizar un algoritmo significa predecir los recursos que requiere el algoritmo. Ocasionalmente, la mayor preocupación son los recursos como la memoria, el ancho de banda de comunicación o el hardware del ordenador, pero la mayoría de las veces es el tiempo computacional lo que queremos medir. Al analizar varios algoritmos candidatos para un problema, se busca identificar el más eficiente. Dicho análisis puede indicar más de un candidato viable, pero a menudo se puede descartar algunos algoritmos más ineficientes en el proceso. Antes de que podamos analizar un algoritmo, debemos tener un modelo de la tecnología de implementación que usaremos, incluido un modelo para los recursos de esa tecnología y sus costos. Durante la mayor parte de este capítulo, se asumirá un modelo de computación genérico de un procesador, usando memoria de acceso aleatorio como nuestra tecnología de implementación y entenderemos que nuestros algoritmos se implementarán como programas de ordenador. En el modelo RAM, las instrucciones se ejecutan una tras otra, sin operaciones concurrentes.

Estrictamente hablando, se debe definir con precisión las instrucciones del modelo RAM y sus costos. Sin embargo, hacerlo sería tedioso y proporcionaría poca información sobre el diseño y análisis de algoritmos. También, debemos tener cuidado de no abusar del modelo RAM. El modelo RAM contiene instrucciones que se encuentran comúnmente en ordenadores reales: aritmética (como sumar, restar, multiplicar, dividir, restar, truncamiento, redondeo), movimiento de datos (cargar, almacenar, copiar) y control (bifurcación condicional e incondicional, llamada de subrutina y retorno). Cada una de estas instrucciones requiere una cantidad constante de tiempo.

Los tipos de datos en el modelo RAM son enteros y de coma flotante (para almacenar números reales). En algunas aplicaciones la precisión es crucial. También asumimos un límite en el tamaño de cada palabra de datos. Por ejemplo, cuando trabajamos con entradas de tamaño n , normalmente asumimos que los enteros están representados por ' $c \cdot \log n$ ' bits para alguna constante ' $c \geq 1$ '. Se requiere ' $c \geq 1$ ' para que cada palabra pueda contener el valor de n , lo que nos permite indexar los elementos de entrada individuales, y restringimos c para que sea una constante de modo que el tamaño de la palabra no crezca arbitrariamente. (Si el tamaño de la palabra pudiera crecer arbitrariamente, podríamos almacenar grandes cantidades de datos en una palabra y operar en todo ello en un tiempo constante, claramente un escenario poco realista).

Los ordenadores reales contienen instrucciones que no se enumeran anteriormente, y tales instrucciones representan un área gris en el modelo de RAM. Por ejemplo, ¿es la exponenciación una instrucción de tiempo constante? En el caso general, no; Se necesitan varias instrucciones para calcular ' x^y ' cuando x e y son números reales. Sin embargo, en situaciones restringidas, la exponenciación es una operación de tiempo constante. Muchos ordenadores tienen una instrucción de "desplazamiento a la izquierda", que en tiempo constante desplaza los bits de un número entero en k posiciones hacia la izquierda. En la mayoría de los ordenadores, mover los bits de un número entero en una posición a la izquierda es equivalente a multiplicar por 2, de modo que mover los bits en k posiciones a la izquierda es equivalente a multiplicar por 2^k siempre que k no sea mayor que el número de bits en una

palabra de ordenador. Intentaremos evitar tales áreas grises en el modelo RAM, pero trataremos el cálculo de 2^k como una operación de tiempo constante cuando k es un número entero positivo lo suficientemente pequeño.

En el modelo de RAM, no intentamos modelar la jerarquía de memoria que es común en los ordenadores contemporáneos. Es decir, no modelamos cachés ni memoria virtual. Varios modelos computacionales intentan explicar los efectos de la jerarquía de memoria, que a veces son significativos en programas reales y en máquinas reales. Los modelos que incluyen la jerarquía de memoria son bastante más complejos que el modelo de RAM, por lo que puede ser difícil trabajar con ellos. Además, los análisis de modelos de RAM suelen ser excelentes predictores del rendimiento en máquinas reales. Analizar incluso un algoritmo simple en el modelo RAM puede ser una tarea compleja. Las herramientas matemáticas necesarias para ello, pueden incluir combinatoria, teoría de la probabilidad, y manipulación algebraica con la capacidad de identificar los términos más significativos en una fórmula. Debido a que el comportamiento de un algoritmo puede ser diferente para cada entrada posible, necesitamos un medio para resumir ese comportamiento en fórmulas simples y fáciles de entender. Aunque disponemos solo de un modelo de máquina para analizar un algoritmo dado, sin embargo, disponemos de muchas opciones para decidir cómo expresar nuestro análisis. Buscamos una forma que sea simple de escribir y manipular, que muestre las características importantes de los requisitos de recursos de un algoritmo y suprima los detalles tediosos.

1.1.9.1 Análisis de ordenación por inserción

El tiempo que tarda el procedimiento INSERTION-SORT depende de la entrada: ordenar mil números toma más tiempo que ordenar 3 números. Además, INSERTION-SORT puede tomar diferentes cantidades de tiempo para ordenar dos secuencias de entrada del mismo tamaño dependiendo de cuán casi ordenadas ya estén. En general, el tiempo que tarda un algoritmo, crece con el tamaño de la entrada, por lo que es tradicional describir el tiempo de ejecución de un programa en función del tamaño de su entrada. Para hacerlo, necesitamos definir los términos "tiempo de ejecución" y "tamaño de entrada" de forma más rigurosa.

La mejor estimación de tamaño de entrada depende del problema que se esté estudiando. Para muchos problemas, como ordenar, o calcular transformadas discretas de Fourier, la medida más natural es el número de elementos en la entrada, por ejemplo, el tamaño de matriz n para ordenar. Para muchos otros problemas, como multiplicar dos números enteros, la mejor medida del tamaño de entrada es *el número total de bits necesarios* para representar la entrada en notación binaria ordinaria. A veces, es más apropiado describir el tamaño de la entrada con dos números en lugar de uno. Por ejemplo, si la entrada a un algoritmo es un gráfico, el tamaño de la entrada se puede describir mediante el número de vértices y aristas en el gráfico. Se indicará qué medida de tamaño de entrada se está utilizando con cada problema que estudiemos.

El **tiempo de ejecución** de un algoritmo de entrada en particular es el número de operaciones primarias o "pasos" ejecutados. Es conveniente definir el tamaño del paso, para que sea lo más independiente posible de la máquina. Por el momento, adoptemos el siguiente punto de vista. Se requiere una cantidad constante de tiempo para ejecutar cada línea de nuestro pseudocódigo. Una línea puede tomar una cantidad de tiempo diferente que otra, pero asumiremos que cada ejecución de la i -ésima línea toma el tiempo c_i , donde c_i es una constante. Este punto de vista está en consonancia con el modelo RAM, y también refleja cómo se implementaría el pseudocódigo en la mayoría de los ordenadores reales².

² Aquí hay algunas sutilezas. Los pasos computacionales que especificamos en inglés son a menudo variantes de un procedimiento que requiere más que una cantidad constante de tiempo. Por ejemplo, más adelante podríamos decir "ordenar los puntos por la coordenada x ", lo que, como veremos, lleva más de una cantidad constante de tiempo. Además, tenga en cuenta que una instrucción que llama a una subrutina lleva un tiempo constante, aunque la subrutina, una vez invocada, puede tardar más. Es decir, separamos el proceso de llamar a la subrutina (pasarle parámetros, etc.) del proceso de ejecutar la subrutina.

En la siguiente discusión, la expresión para el tiempo de ejecución de INSERTIONSORT evolucionará a partir de una fórmula desordenada que usa toda la declaración de costes ' c_i ' a una notación mucho más simple que es más concisa y fácil de manipular. Esta notación más simple también facilitará la determinación de si un algoritmo es más eficiente que otro. Comenzamos presentando el procedimiento INSERTION-SORT con el "costo" de tiempo de cada instrucción y el número de veces que se ejecuta cada instrucción. Para cada ' $j = 2, 3, \dots, n$ ', donde $n = A.length$, tomamos t_j como el número de veces que se ejecuta la prueba del ciclo '**while**' en la línea 5 para ese valor de j . Cuando un bucle **for** o **while** sale de la forma habitual (es decir, debido a la evaluación de la condición en el encabezado del bucle), esta evaluación, se ejecuta una vez más que el cuerpo del bucle. Suponemos que los comentarios no son declaraciones ejecutables, por lo que no toman tiempo.

INSERTION-SORT(A) /	Coste	tiempo
For $j = 2$ to $A.length$	C1	n
$Key = A[j]$	C2	$n - 1$
// Inserte $A[j]$ en la secuencia ordenada $A[1, 2, 3, \dots, j-1]$	0	$n - 1$
$i = j - 1$	C4	$n - 1$
while $i > 0$ and $A[i] > key$	C5	$\sum_{j=2}^n t_j$
$A[i + 1] = A[i]$	C6	$\sum_{j=2}^n (t_j - 1)$
$i = i - 1$	C7	$\sum_{j=2}^n (t_j - 1)$
$A[i + 1] = key$	C8	$N - 1$

El tiempo de ejecución del algoritmo es la suma de los tiempos de ejecución de cada instrucción ejecutada; una declaración que toma ' c_i ' pasos para ejecutarse y ejecuta n veces contribuirá con ' $c_i n$ ' al tiempo total de ejecución³. Para calcular $T(n)$, el tiempo de ejecución de INSERTION-SORT en una entrada de n valores, sumamos los productos del costo y los tiempos columnas, obteniendo

$$T(n) = c_1 n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n - 1)$$

Incluso para entradas de un tamaño determinado, el tiempo de ejecución de un algoritmo puede depender de qué entrada de ese tamaño se proporcione. Por ejemplo, en INSERTION-SORT, el mejor caso ocurre si la matriz ya está ordenada. Para cada $j = 2, 3, \dots, n$, entonces encontramos que ' $A[j] \leq key$ ' en la línea 5 cuando i tiene su valor inicial de $j - 1$. Así ' $t_j = 1$ ' para ' $j = 2, 3, \dots, n$ ', y el mejor tiempo de ejecución es:

$$T(n) = c_1 n + c_2(n - 1) + c_4(n - 1) + c_5(n - 1) + c_8(n - 1) = (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8)$$

Podemos expresar este tiempo de ejecución como $(an + b)$ para las constantes a y b que dependen de los costos del enunciado c_i , por tanto, es una **función lineal de n** .

³ Esta característica no es necesariamente válida para un recurso como la memoria. Una declaración que hace referencia a m palabras de la memoria y se ejecuta n veces no necesariamente hace referencia a $m \cdot n$ palabras distintas de la memoria.

Si la matriz está en orden inverso, es decir, en orden decreciente, se produce el peor de los casos. Debemos comparar cada elemento $A[j]$ con cada elemento en toda la submatriz ordenada $A[1, 2, \dots, j-1]$, y así $t_j = j$ para $j = 2, 3, \dots, n$. Tenemos entonces que:

$$\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$$

Y

$$\sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$$

Encontramos que, en el peor de los casos, el tiempo de ejecución de INSERTION-SORT es :

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) + c_6 \frac{n(n-1)}{2} + c_7 \frac{n(n-1)}{2} + c_8(n-1) = \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + (c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8) n - (c_2 + c_4 + c_5 + c_8)$$

Podemos expresar esto tiempo de ejecución en el peor de los casos como:

$a \cdot n^2 + b \cdot n + c$ para las constantes a , b y c que nuevamente dependen de los costos de declaración ' c_i '; por tanto, es una **función cuadrática de n** . Por lo general, en la "ordenación por inserción", el tiempo de ejecución de un algoritmo se fija para una entrada determinada, aunque existen algunos algoritmos "aleatorizados" interesantes cuyo comportamiento puede variar incluso para una entrada fija.

1.1.9.2 Análisis del peor caso y caso promedio

En nuestro análisis de "ordenación por inserción", hemos analizado tanto el caso mejor, en el que la matriz de entrada ya está ordenada, como el peor de los casos, en el que la matriz de entrada se ordenó al revés. Sin embargo, normalmente durante el resto de esta exposición, nos concentraremos en encontrar sólo el tiempo de ejecución del peor de los casos, es decir, el tiempo de ejecución más largo para cualquier entrada de tamaño n . Damos tres razones para esta orientación.

- El tiempo de ejecución en el peor de los casos de un algoritmo nos da un límite superior en el tiempo de ejecución para cualquier entrada. Saberlo proporciona una garantía de que el algoritmo nunca tardará más.
- Para algunos algoritmos, el peor de los casos ocurre con bastante frecuencia. Por ejemplo, al buscar en una base de datos un dato en particular, el peor de los casos del algoritmo de búsqueda ocurre cuando la información no está presente en la base de datos. En algunas aplicaciones, las búsquedas de información inexistente pueden ser frecuentes.
- El "caso medio" suele ser tan malo como el peor de los casos. Suponga que elegimos aleatoriamente n números y aplicamos ordenación por inserción. ¿Cuánto tiempo se tarda en determinar en qué lugar de la submatriz $A[1 \dots j-1]$, se inserta el elemento $A[j]$? En promedio, la mitad de los elementos de $A[1 \dots j-1]$, son menores que $A[j]$ y la mitad de los elementos son mayores. Por lo tanto, en promedio, verificamos la mitad de la submatriz $A[1 \dots j-1]$, por lo que t_j es aproximadamente $j/2$. El tiempo medio de ejecución resultante resulta ser una función cuadrática del tamaño de entrada, al igual que el tiempo de ejecución del peor de los casos.

En algunos casos particulares, nos interesará el **tiempo de ejecución promedio** de un algoritmo; El alcance del análisis de casos promedio es limitado, porque puede que no sea

evidente qué constituye una entrada "promedio" para un problema en particular. A menudo, asumiremos que todas las entradas de un tamaño dado son igualmente probables. En la práctica, esta suposición puede no darse, pero a veces podemos usar un algoritmo aleatorio, que toma decisiones al azar, para permitir un análisis probabilístico y producir un tiempo de ejecución esperado.

1.1.10 Orden de crecimiento

Usamos algunas abstracciones simplificadoras para facilitar el análisis del procedimiento INSERTIONSORT. Primero, ignoramos el costo real de cada declaración, usando las constantes c_i para representar estos costos. Luego, observamos que incluso estas constantes nos dan más detalles de los que realmente necesitamos: expresamos el tiempo de ejecución en el peor de los casos como $a \cdot n^2 + b \cdot n + c$ para algunas constantes a , b y c que dependen de los costos del enunciado c_i . Por lo tanto, ignoramos no solo los costos de declaración reales, sino también los costos abstractos c_i .

Ahora haremos una abstracción más simplificadora: **es la tasa de crecimiento**, o el **orden de crecimiento**, del tiempo de ejecución lo que realmente nos interesa. Por lo tanto, consideramos solo el término principal de una fórmula (por ejemplo, $a \cdot n^2$), ya que los términos de orden inferior son relativamente insignificantes para valores grandes de n . También ignoramos el coeficiente constante del término principal, ya que los factores constantes son menos significativos que la tasa de crecimiento para determinar la eficiencia computacional para grandes entradas. Para la ordenación por inserción, cuando ignoramos los términos de orden inferior y el coeficiente constante del término principal, nos queda el factor de n^2 del término principal. Escribimos que la ordenación por inserción tiene un tiempo de ejecución en el peor de los casos de $\theta(n^2)$ (pronunciado "theta de n-cuadrado").

Por lo general, consideramos que un algoritmo es más eficiente que otro si su peor tiempo de ejecución tiene un orden de crecimiento menor. Debido a factores constantes y términos de orden inferior, un algoritmo cuyo tiempo de ejecución tiene un orden de crecimiento más alto puede llevar menos tiempo para entradas pequeñas que un algoritmo cuyo tiempo de ejecución tiene un orden de crecimiento inferior. Pero para entradas lo suficientemente grandes, un algoritmo $\theta(n^2)$, por ejemplo, se ejecutará más rápidamente en el peor de los casos que un algoritmo $\theta(n^3)$.

1.1.11 Diseño de algoritmos

Podemos elegir entre una amplia gama de técnicas de diseño de algoritmos. Para la ordenación por inserción, usamos un enfoque **incremental**: habiendo ordenado la submatriz $A[1 \dots j - 1]$, insertamos el elemento individual $A[j]$ en su lugar apropiado, produciendo la submatriz ordenada $A[1 \dots j]$. En esta sección, examinamos un enfoque de diseño alternativo, conocido como "divide y vencerás". Lo usaremos para diseñar un algoritmo de ordenación cuyo tiempo de ejecución en el peor de los casos, sea mucho menor que el de la ordenación por inserción.

1.1.11.1 El enfoque de divide y vencerás

Muchos algoritmos útiles tienen una estructura **recursiva**: para resolver un problema dado, se llaman a sí mismos recursivamente una o más veces para tratar subproblemas muy relacionados. Estos algoritmos suelen seguir un enfoque de "divide y vencerás": dividen el problema en varios subproblemas que son similares al problema original, pero de menor tamaño, resuelven los subproblemas de forma recursiva y luego combinan estas soluciones para crear una solución al problema original.

Plan Básico.

- Dividir la matriz en dos mitades.
- Ordenar de forma recursiva cada mitad.
- Fusionar dos mitades.

Entrada	M	E	R	G	E	S	O	R	T	E	X	A	M	P	L	E
Ordenar mitad izquierda	E	E	G	M	O	R	R	S	T	E	X	A	M	P	L	E
Ordenar mitad derecha	E	E	G	M	O	R	R	S	A	E	E	L	M	P	T	X
Fusionar resultados	A	E	E	E	E	G	L	M	M	O	P	R	R	S	T	X

Descripción general de Mergesort

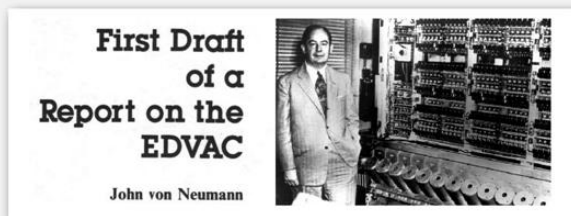


Figura 1.4 Evolución de la clasificación de Mergesort.

El paradigma “divide y vencerás” implica tres pasos en cada nivel de la recursividad:

Divide el problema en varios subproblemas que son instancias más pequeñas del mismo problema.

Resuelve los subproblemas de forma recurrente. Sin embargo, si los tamaños de los subproblemas son lo suficientemente pequeños, se resuelven los subproblemas de la manera más directa.

Combina las soluciones de los subproblemas en la solución del problema original. El algoritmo de ordenación por **mergesort** sigue de cerca el paradigma de “divide y vencerás”. Intuitivamente, funciona de la siguiente manera.

Dividir: Divide la secuencia de n elementos para clasificar en dos subsecuencias de $(n / 2)$ elementos cada una.

Superar: Ordena las dos subsecuencias de forma recursiva utilizando la clasificación por combinación (**mergesort**).

Combinar: Combina las dos subsecuencias ordenadas para producir la respuesta ordenada.

La recursividad “llega a su fin” cuando la secuencia a ordenar tiene longitud 1, en cuyo caso no hay trabajo por hacer, ya que cada secuencia de longitud 1 ya está ordenada.

Objetivo. Dados dos submatrices ordenadas $a[lo]$ hasta $a[mid]$ y $a[mid + 1]$ hasta $a[hi]$, reemplazar con la submatriz ordenada $a[lo]$ a $a[hi]$.

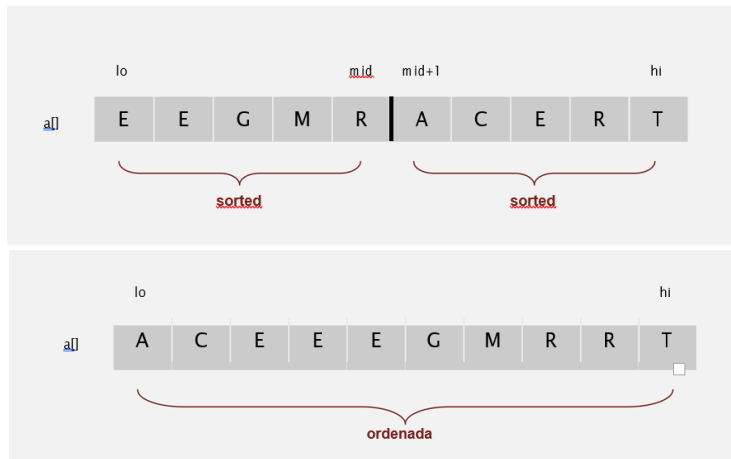


Figura 1.5 Paso final Mergesort..

La **operación clave** del algoritmo de clasificación mergesort es la combinación de dos secuencias ordenadas en el modo "combinación". Se unen llamando a un procedimiento auxiliar **MERGE(A, p, q, r)**, donde A es una matriz y p, q, y r son índices en la matriz tal **que** $p \leq q < r$. El procedimiento asume que las sub matrices **A[p..q]** y **A[q + 1...r]** están ordenadas. Las fusiona para formar una única submatriz ordenada que reemplaza a la submatriz actual **A[p...r]**.

El procedimiento MERGE lleva tiempo $\theta(n)$, donde $n = r - p + 1$ es el número total de elementos que se mezclan, y funciona de la siguiente manera: Tomando el caso de juego de cartas, supongamos que tenemos dos pilas de cartas boca arriba en una mesa. Cada pila está ordenada, con las cartas más bajas en la parte superior. Deseamos fusionar las dos pilas en una sola de salida ordenada, que debe estar boca abajo sobre la mesa. Nuestro paso básico consiste en elegir la más pequeña de las dos cartas en la parte superior de las pilas boca arriba, sacarla de su pila (lo que expone una nueva carta superior) y colocar esta carta boca abajo en la pila de salida. Repetimos este paso hasta que una pila de entrada esté vacía, momento en el que simplemente tomamos como pila de entrada la restante y la vamos colocando boca abajo en la pila de salida. Computacionalmente, cada paso básico toma un tiempo constante, ya que estamos comparando solo las dos cartas superiores. Dado que realizamos como máximo n pasos básicos, la combinación lleva $\theta(n)$ tiempo.

El siguiente pseudocódigo implementa la idea anterior, pero con un giro adicional que evita tener que verificar si alguna pila está vacía en cada paso básico. Colocamos en la parte inferior de cada pila una tarjeta **centinela**, que contiene un valor especial que usamos para simplificar nuestro código. Aquí, usamos ∞ como valor centinela, de modo que siempre que una carta con ∞ esté expuesta, no puede ser la carta más pequeña a menos que ambas pilas tengan sus cartas centinela expuestas. Pero una vez que eso sucede, todas las cartas que no son centinelas ya se han colocado en la pila de salida. Como sabemos de antemano que exactamente $r - p + 1$ cartas se colocarán en la pila de salida, podemos detenernos una vez que hayamos realizado tantos pasos básicos.

```

MERGE(.A, p, q, r)
1       $n_1 = q - p + 1$ 
2       $n_2 = r - q$ 
3      Sean  $L[1....n_1 + 1]$  y  $R[1....n_2 + 1]$  nuevas matrices

```

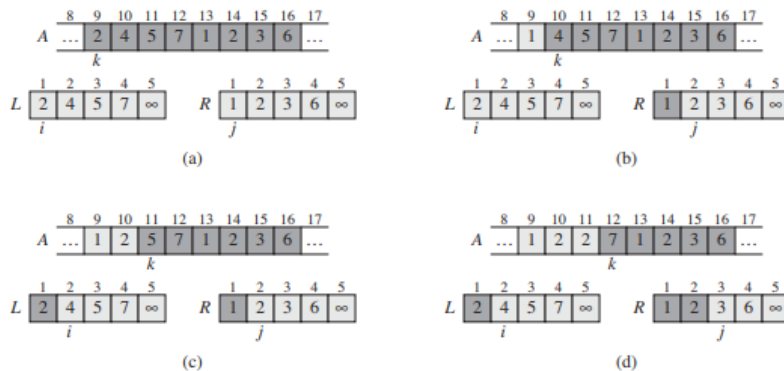
```

4      for i=1 to n1
5      L[i] = A[p + i - 1]
6      for j=1 to n2
7      R[j] = A[q + j]
8      L[n1 + 1] = ∞
9      R[n2 + 1] = ∞
10     i = 1
11     j = 1
12     for k=p to r
13         if L[i] ≤ R[j]
14             A[k] ≈ L[i]
15             i ≈ i + 1
16         else A[k] ≈ R[j]
17             j = j + 1

```

En detalle, el procedimiento MERGE funciona como sigue: en la línea 1 calcula la longitud n_1 de la submatriz $A[p...q]$, y la línea 2 calcula la longitud n_2 de la submatriz $A[q+1...r]$. Se crean las matrices L y R ("izquierda" y "derecha"), de longitudes $n_1 + 1$ y $n_2 + 1$, respectivamente, en la línea 3; la posición adicional en cada matriz contendrá el centinela. El bucle 'for' de las líneas 4-5 copia la submatriz $A[p...q]$ en $L[1...n_1]$, y el bucle 'for' de las líneas 6-7 copia la submatriz $A[q+1...r]$ en $R[1...n_2]$. Las líneas 8 a 9 colocan a los centinelas en los extremos de las matrices L y R. Las líneas 10 a 17, ilustradas en el código, realizan los pasos básicos de $r - p + 1$ manteniendo el siguiente ciclo invariante:

Al comienzo de cada iteración del ciclo 'for' de las líneas 12-17, la submatriz $A[p..k-1]$ contiene los $k - p$ elementos más pequeños de $L[1...n_1 + 1]$ y $R[1...n_2 + 1]$, ordenados. Además, $L[i]$ y $R[j]$ son los elementos más pequeños de sus matrices que no se han vuelto a copiar en A.



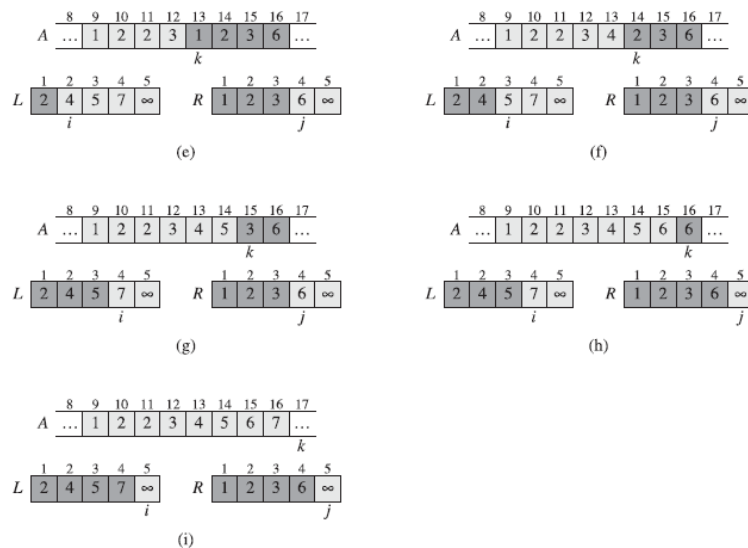
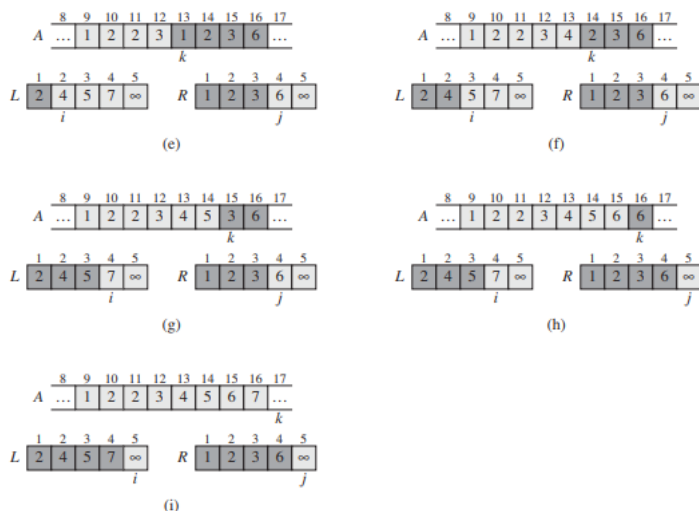


Figura 1.6 La operación de las líneas 10-17 en la llamada $MERGE(A, 9, 12, 16)$, cuando la submatriz $A[9...16]$ contiene la secuencia $\{2, 4, 5, 7, 1, 2, 3, 6\}$. Después de copiar e insertar centinelas, la matriz L contiene $\{2, 4, 5, 7, \infty\}$, y la matriz R contiene $\{1, 2, 3, 6, 1, \infty\}$. Las posiciones ligeramente sombreadas en A contienen sus valores finales, y las posiciones ligeramente sombreadas en L y R contienen valores que aún no se han vuelto a copiar en A . En conjunto, las posiciones ligeramente sombreadas siempre comprenden los valores originalmente en $A[9...16]$, junto con los dos centinelas. Las posiciones muy sombreadas en A contienen valores que se copiarán, y las posiciones muy sombreadas en L y R contienen valores que ya se han copiado de nuevo en A . (a) - (h) Las matrices A , L y R , y sus respectivas índices k , i , y j antes de cada iteración del ciclo de las líneas 12-17 (i) Las matrices e índices al final. En este punto, la submatriz en $A[9...16]$ está ordenada, y los dos centinelas en L y R son los únicos dos elementos en estas matrices que no se han copiado en A .

Debemos demostrar que este ciclo invariante se mantiene antes de la primera iteración del ciclo **for** de las líneas 12-17, que cada iteración del ciclo mantiene el invariante y que el invariante proporciona una propiedad útil para mostrar la corrección cuando el ciclo termina.

Inicialización: antes de la primera iteración del ciclo, tenemos $k = p$, de modo que la submatriz $A[p...k - 1]$ está vacía. Esta submatriz vacía contiene los $k - p = 0$ elementos más pequeños de L y R , y dado que $i = j = 1$, tanto $L[i]$ como $R[j]$ son los elementos más pequeños de sus matrices, que no se han vuelto a copiar en A .



Mantenimiento: Para ver que cada iteración mantiene el bucle invariable, vamos a suponer que $L[i] \leq R[j]$. Entonces $L[i]$ es el elemento más pequeño que aún no se ha copiado en A. Dado que $A[p...k-1]$ contiene los elementos $k-p$ más pequeños, después de que la línea 14 copia $L[i]$ en $A[k]$, el submatriz $A[p...k]$ contendrá los $k-p-1$ elementos más pequeños. Incrementando k (en la actualización del bucle 'for') e i (en la línea 15) prepara el bucle para la siguiente iteración. Si en su lugar $L[i] > R[j]$, a continuación, las líneas 16-17 realizan la acción adecuada para la siguiente iteración del bucle.

Terminación: En la terminación, $k = r + 1$. Por el bucle invariable, la submatriz $A[p...k-1]$, que es $A[p...r]$, contiene los $k-p = r-p+1$ elementos más pequeños de $L[1...n_1+1]$ y $R[1...n_2+1]$, en orden. Las matrices L y R juntas contienen $n_1 + n_2 + 2 = r - p + 3$ elementos. Todos menos los dos más grandes han sido copiados de nuevo en A, y estos dos elementos más grandes son los centinelas.

Para ver que el procedimiento MERGE se ejecuta en tiempo $\theta(n)$, donde $n = r - p + 1$, observe que cada una de las líneas 1-3 y 8-11 toma un tiempo constante,⁴ los bucles for de las líneas 4-7 toman tiempo $\theta(n_1 + n_2) = \theta(n)$ y hay n iteraciones del bucle 'for' de las líneas 12-17, cada una de las cuales toma un tiempo constante.

Ahora podemos usar el procedimiento MERGE como una subrutina en el algoritmo de clasificación por fusión. El procedimiento **MERGE-SORT(A, p, r)** ordena los elementos de la submatriz $A[p...r]$. Si $p \geq r$, la submatriz tiene como máximo un elemento y, por lo tanto, ya está ordenada. De lo contrario, el paso de división simplemente calcula un índice q que divide $A[p...r]$, en dos submatrices: $A[p...q]$, que contiene $(n/2)$ elementos, y $A[q+1...r]$, que contiene $(n/2)$ elementos⁵.

MERGE-SORT.(A,p,r)

1 If $p < r$

⁴ Veremos en el apartado 1.3 cómo interpretar formalmente las ecuaciones que contienen la notación $\theta(n)$.

⁵ La expresión $\lfloor x \rfloor$ denota el menor entero mayor o igual a x , y $\lceil x \rceil$ denota el mayor entero menor o igual a x . Estas notaciones se definen en el apartado 1.3. La forma más fácil de verificar que el ajuste $q = \lfloor (p+r)/2 \rfloor$ produce submatrices $A[p...q]$ y $A[q+1...r]$ de tamaños $\lfloor n/2 \rfloor$ y $\lfloor n/2 \rfloor$ respectivamente, es examinar los cuatro casos que surgen dependiendo de si cada uno de p y r es par o impar.

```

2         q = [(p + r) / 2]
3     MERGE-SORT(A, p, q)
4     MERGE-SORT(A q + 1, r)
5     MERGE(A, p, q, r)

```

Para ordenar toda la secuencia $A = (A[1], A[2], \dots, A[n])$, hacemos la llamada inicial $MERGE-SORT(A, 1, A.length)$, donde una vez más $A.length = n$. La figura 1.7 ilustra el funcionamiento del procedimiento de abajo hacia arriba cuando n es una potencia de 2. El algoritmo consiste en combinar pares de secuencias de 1 elemento para formar secuencias ordenadas de longitud 2, combinando pares de secuencias de longitud 2 para formar secuencias ordenadas de longitud 4, y así sucesivamente, hasta que dos secuencias de longitud $n/2$ se fusionen para formar la secuencia final ordenada de longitud n .

Mergesort: implementación de Java:

```

private static void merge(Comparable[] a, Comparable[] aux, int lo, int mid, int hi) {

    assert isSorted(a, lo, mid); // precondition: a[lo..mid] sorted
    assert isSorted(a, mid+1, hi); // precondition: a[mid+1..hi] sorted
    for (int k = lo; k <= hi; k++) copy
        aux[k] = a[k];

    int i = lo, j = mid+1;
    for (int k = lo; k <= hi; k++) merge
    {
        if (i > mid) a[k] = aux[j++];
        else if (j > hi) a[k] = aux[i++];
        else if (less(aux[j], aux[i])) a[k] = aux[j++];
        else a[k] = aux[i++];
    }
    assert isSorted(a, lo, hi); // postcondition: a[lo..hi] sorted
}

```

1.1.11.2 Análisis de algoritmos de divide y vencerás

Cuando un algoritmo contiene una llamada recursiva a sí mismo, a menudo podemos describir su tiempo de ejecución mediante una ecuación de recurrencia o recurrente, que describe el tiempo de ejecución general en un problema de tamaño n , en términos del tiempo de ejecución en entradas más pequeñas. Luego, podemos usar herramientas matemáticas para resolver la recurrencia y proporcionar límites sobre el rendimiento del algoritmo.

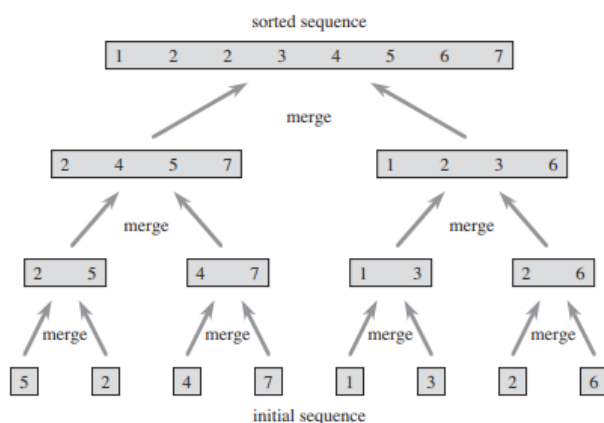


Figura 1.7 La operación de clasificación por fusión en la matriz $A\{5, 2, 4, 7, 1, 3, 2, 6\}$. Las longitudes de las secuencias ordenadas que se fusionan aumentan a medida que el algoritmo avanza de abajo hacia arriba.

La repetición del tiempo de ejecución de un algoritmo de divide y vencerás se sale de los tres pasos del paradigma básico. Como antes, dejamos que $T(n)$ sea el tiempo de ejecución de un problema de tamaño n . Si el tamaño del problema es lo suficientemente pequeño, digamos $n \leq c$ para alguna constante c , la solución sencilla toma un tiempo constante, que escribimos como $\theta(1)$. Suponga que la división del problema produce x subproblemas, cada uno de los cuales es $1/y$ del tamaño del original. (Para la ordenación mergesort, tanto x como y son 2, pero veremos muchos algoritmos de divide y vencerás en los que $x \neq y$.) Se necesita tiempo $T(n/y)$ para resolver un subproblema de tamaño n/y , por lo que toma tiempo $xT(n/y)$ resolver uno de ellos. Si tomamos $D(n)$ el tiempo para dividir el problema en subproblemas y $C(n)$ el tiempo para combinar las soluciones de los subproblemas en la solución del problema original, obtenemos la recurrencia

$$Tn = \left\{ \begin{array}{ll} \theta(1) & \text{Si } n \leq c \\ aT\left(\frac{n}{b}\right) + D(n) + C(n) & \text{En otro caso} \end{array} \right\}$$

1.1.11.3 Análisis de mergesort.

Aunque el pseudocódigo para MERGE-SORT funciona correctamente cuando el número de elementos incluso no es par, nuestro análisis basado en la recurrencia se simplifica si asumimos que el tamaño del problema original es una potencia de 2. Cada paso de división produce dos subsecuencias de tamaño exactamente $(n/2)$. Este supuesto no afecta el orden de crecimiento de la solución a la recurrencia. Mergesort en un solo elemento lleva un tiempo constante. Cuando tenemos $n > 1$ elementos, desglosamos el tiempo de ejecución de la siguiente manera:

Dividir: el paso de división solo calcula la mitad de la submatriz, lo que lleva un tiempo constante. Por lo tanto, $D(n) = \theta(1)$

Conquistar: Resolvemos de forma recursiva dos subproblemas, cada uno de tamaño: $(n/2)$, que aporta $2T(n/2)$ al tiempo de ejecución.

Combinar: Ya hemos notado que el procedimiento MERGE en una submatriz de n elementos toma tiempo $\theta(n)$ y así $C(n) = \theta(n)$.

Cuando agregamos las funciones $D(n)$ y $C(n)$ para el análisis de mergesort, estamos agregando una función que es $\theta(n)$ y una función que es, $\theta(1)$. Esta suma es una función lineal de n , es decir $\theta(n)$. Al agregarlo al término $2T(n/2)$ del paso "conquistar", se obtiene la recurrencia del tiempo de ejecución del peor caso $T(n)$ del mergesort:

$$T(n) \approx \begin{cases} \theta(1) & \text{Si } n=1 \\ 2T(n/2) + \theta(n) & \text{Si } n>1 \end{cases} \quad (2.1)$$

Debido a que la función logaritmo crece más lentamente que cualquier función lineal, para entradas lo suficientemente grandes, mergesort, con su tiempo de ejecución $\theta(n \log n)$ supera al ordenamiento por inserción, cuyo tiempo de ejecución es, $\theta(n^2)$ en el peor de los casos. Necesitamos el teorema maestro para comprender intuitivamente por qué la solución de la recurrencia (2.1) es $T(n) = \theta(n \log n)$ Reescribamos la recurrencia (2.1) como

$$T(n) \approx \begin{cases} c & \text{Si } n=1 \\ 2T(n/2) + cn & \text{Si } n>1 \end{cases} \quad (2.2)$$

donde la constante c representa el tiempo necesario para resolver problemas de tamaño 1, así como el tiempo por elemento de matriz, de los pasos de dividir y combinar⁶.

La figura 1.8 muestra cómo podemos resolver la recurrencia (2.2). Por conveniencia, asumimos que n es una potencia exacta de 2. La parte (a) de la figura muestra $T(n)$, que expandimos en la parte (b) en un árbol equivalente que representa la recurrencia.

El término cn es la raíz (el costo incurrido en el nivel superior de recursividad), y los dos subárboles de la raíz son las dos partes recursivas más pequeñas $T(n/2)$. La parte (c) muestra que este proceso avanzó un paso más al expandir $T(n/2)$. El costo incurrido en cada uno de los dos subnodos en el segundo nivel de recursividad es $cn/2$. Continuamos expandiendo cada nodo en el árbol dividiéndolo en sus partes constituyentes según lo determinado por la recurrencia, hasta que los tamaños de los problemas bajen a 1, cada uno con un costo de c . La parte (d) muestra el **árbol de recursividad** resultante.

A continuación, sumamos los costos en cada nivel del árbol. El nivel superior tiene un costo total cn , el siguiente nivel inferior tiene un costo total $c(n/2) + c(n/2) = cn$, el nivel posterior tiene un costo total $c(n/4) + c(n/4) + c(n/4) + c(n/4) = cn$, y así sucesivamente. En general, el nivel i debajo de la parte superior tiene 2^i nodos, cada uno contribuyendo con un costo de $c(n/2^i)$, de modo que el i -ésimo nivel debajo de la parte superior tiene un costo total $2^i c(n/2^i) = cn$. El nivel inferior tiene n nodos, cada uno de los cuales contribuye con un costo de c , para un costo total de cn .

El número total de niveles del árbol de recursividad en la Figura 1,8 es $\log n + 1$, donde n es el número de hojas, correspondiente al tamaño de entrada. Un argumento inductivo informal justifica esta afirmación. El caso base ocurre cuando $n = 1$, en cuyo caso el árbol tiene un solo nivel. Dado que $\log 1 = 0$, tenemos que $\log n + 1$ da el número correcto de niveles. Ahora suponga como hipótesis inductiva que el número de niveles de un árbol de recursividad con 2^i hojas es $\lg 2^i + 1 = i + 1$ (ya que, para cualquier valor de i , tenemos que $\lg 2^i = i$). Debido a que asumimos que el tamaño de entrada es una potencia de 2, el siguiente tamaño de entrada a considerar es 2^{i+1} . Un árbol con $n = 2^{i+1}$ hojas, tiene un nivel más que un árbol con 2^i hojas, por lo que el número total de niveles es $(i + 1) + 1 = \log 2^{i+1} + 1$. Para calcular el costo total representado por la recurrencia (2.2), simplemente sumamos los costos de todos los niveles. El árbol de recursividad tiene $(\log n + 1)$ niveles, cada uno con un costo cn , por un costo total

⁶ Es poco probable que la misma constante represente exactamente tanto el tiempo para resolver problemas de tamaño 1 como el tiempo por elemento de matriz de los pasos de dividir y combinar. tiempos y entendiendo que nuestra recurrencia da un límite inferior en el tiempo de ejecución. Ambos límites están en el orden de $n \log n$ y, en conjunto, dan un $\theta(n \log n)$ tiempo de ejecución. Podemos solucionar este problema dejando que c sea el mayor de estos tiempos y entendiendo que nuestra recurrencia da un límite superior en el tiempo de ejecución, o dejando que c sea el menor de estos.

de $cn(\log n + 1) = cn \log n + cn$. Ignorar el término de orden inferior y la constante c da el resultado deseado de $\theta(n \log n)$.ⁱⁱ

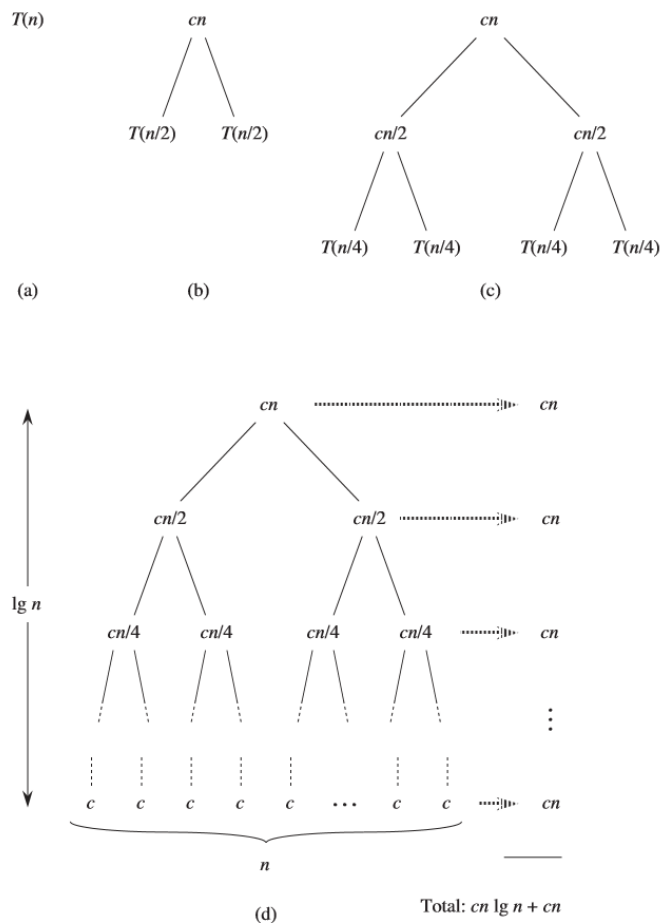


Figura 1,8 Cómo construir un árbol de recursividad para la recurrencia $T(n) = 2T(n/2) + cn$. La parte (a) muestra $T(n)$, que se expande progresivamente en (b) - (d) para formar el árbol de recursividad. El árbol completamente expandido en la parte (d) tiene niveles de $(\log n + 1)$ (es decir, tiene una altura de $\log n$, como se indica), y cada nivel aporta un costo total de cn . El costo total, por lo tanto, es $cn * \log n + cn$, que es $\theta(n * \log n)$

Corrección de los algoritmos recursivos

Para probar la corrección de un algoritmo recursivo:

- Pruébalo por inducción sobre el "tamaño" del problema a resolver (p. ej., tamaño del fragmento de matriz, número de bits en un número entero, etc.)
- La base de la recursividad es la base de la inducción.
- Necesidad de demostrar que a las llamadas recursivas se les asignan subproblemas, es decir, sin recursividad infinita (a menudo trivial).

- Paso inductivo: suponga que las llamadas recursivas funcionan correctamente, y utilice esta suposición para probar que la llamada actual funciona correctamente.

Mergesort: implementación Java recursiva.

```

    public class Merge
    {private static void merge(...)
      { /* as before */ }
    private static void sort(Comparable[] a, Comparable[] aux, int lo, int hi) {
      if (hi <= lo) return;
      int mid = lo + (hi - lo) / 2;
      sort(a, aux, lo, mid);
      sort(a, aux, mid+1, hi);
      merge(a, aux, lo, mid, hi);
    }

    public static void sort(Comparable[] a)
    {aux = new Comparable[a.length];
      sort(a, aux, 0, a.length - 1);
    }
  }
}

```

1.1.11.4 Mergesort: análisis empírico

Estimaciones de tiempo de ejecución:

- Un ordenador portátil ejecuta 108 comparaciones por segundo.
- Un supercomputador ejecuta 1012 comparaciones por segundo.

computer	insertion sort (N^2)			mergesort ($N \log N$)		
	Miles	millones	Billones	Miles	Millones	Billones
Portátil	instantáneamente	2.8 horas	317 años	instantáneamente	1 segundo	18 min
super	instantáneamente	1 segundos	1 semana	instantáneamente	instantáneamente	instantáneamente

Figura 1.9 Comparativa entre algoritmos insertion y mergesort..

1.1.11.5 Divide y vencerás: prueba por imagen.

Proposición: Si $D(N)$ satisface $D(N) \leq 2D(N/2) + N$ para $N > 1$, con $D(1) = 0$, luego $D(N) \leq N \lg N$.

Pf 1. [suponiendo que N es una potencia de 2]

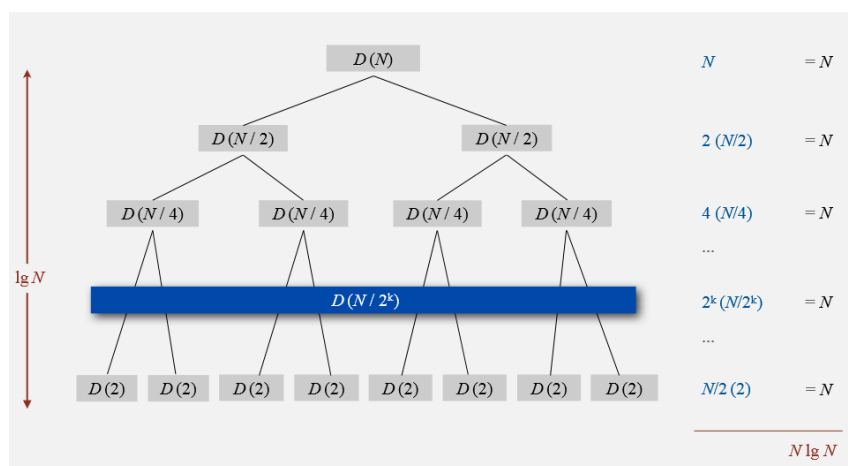


Figura 1.10 Prueba por imagen de "Divide y vencerás".

1.2 Técnicas para medir y estimar el tamaño del software a partir de la complejidad de sus funcionalidades.

1.2.1 Introducción.

La elaboración de estimaciones de proyectos de software, que consiste en identificar cuantas horas hombre o jornadas tomará el implementar un sistema informático, ha sido y continuará siendo uno de los aspectos cruciales en el desarrollo de software en todas las organizaciones.

Según estadísticas del sector industrial, aproximadamente el 40% de proyectos de software son cancelados debido a estimaciones parciales o completamente erróneas, por ello es necesario aplicar una metodología en la elaboración de la estimación.

El método de 5 pasos para elaborar estimaciones de proyectos de software consiste en: Revisar los requerimientos y el alcance, Dimensionar el trabajo a realizar (desglose de trabajo), Elaborar una primera estimación base, Identificar y analizar los riesgos, y por último Validar y revisar la estimación.

1.2.2 Pasos para elaborar estimaciones de proyectos de software

A.- Revisar los requerimientos y el alcance

- Primero se establecen las expectativas de los interesados en cuanto a la exactitud de la estimación:
 - Se expresan en un rango (tales como: 50%, 10% o 1%).
 - Cuanta más información disponible, mayor exactitud.
 - De tal forma, no se puede esperar una estimación de un % determinado cuando los requerimientos son vagos o no se han analizado todavía lo suficiente.
- Luego se debe obtener los requerimientos funcionales:
 - ¿Existe una especificación de diseño funcional o una solicitud formal de requerimientos ?
 - Si se trabaja con una metodología ágil, se deberá hacer énfasis en entrevistas y en comprender fielmente los requerimientos para poder estimarlos.
 - ¿Están todos los puntos claramente definidos o existen ambigüedades por resolver?
 - Cuantas más ambigüedades se resuelvan, mayor exactitud tendrá la estimación.
- Prestar atención a los requerimientos no funcionales:
 - Tales como: de seguridad, tiempo de respuesta, esfuerzo de mantenimiento, entre otros.
 - Su no observancia puede hacer fracasar un proyecto.
- Se debe investigar sobre la mayor cantidad de detalles técnicos, de dependencias externas y requerimientos originales del negocio que componen la estimación.
- Se debe establecer claramente qué funcionalidades estarán incluidas en la estimación y definir explícitamente cuáles no.
- Hacer una evaluación de estos antecedentes, por ejemplo:

- ¿Se asume que se va a reutilizar el software?, ¿ese software reutilizado se encuentra en producción?, ¿fue testeado adecuadamente?
- ¿Se está asumiendo que estarán disponibles los recursos, por ejemplo, ciertos recursos expertos?, ¿qué ocurriría con la estimación si no estuvieran disponibles y tuviera que buscarse personas (con la correspondiente curva de aprendizaje)?
- Debe evaluarse cualquier aspecto fuente de incertidumbre.

B.- Dimensionar el trabajo a realizar (desglose de trabajo)

- En este paso, nos enfocamos en el “Qué” vamos a estimar.
- Se debe identificar los componentes que se van a desarrollar, la profundidad y detalle dependerá del grado de exactitud que se quiera dar a la estimación.
- Si se quiere tener detalle, se debe identificar: las Tablas y procedimientos de bases de datos, los servicios web, las funciones y procedimientos, pantallas, e interfaces con otros componentes, etc.
- Este dimensionamiento no significa definir en detalle cada componente, no se espera definir con precisión sus entradas, salidas, posibles valores, líneas de código, etc. Sin embargo, si se espera poder identificarlos individualmente y asignarles niveles de complejidad.
- Esta es la dimensión que más influye en la estimación y, por tanto, la fase más importante para lograr una estimación acertada.
- Debe identificarse el nuevo software a desarrollar, pero hay que compatibilizarlo con el existente, estudiar modificaciones de componentes, realizar pruebas para ver si funcionan y otros aspectos que deban tomarse en cuenta.
- Para identificar los componentes, se puede recurrir a la opinión de los expertos, a una metodología formal (desglosar por funciones o por casos de uso por ejemplo), o un dimensionamiento estadístico si disponemos de información histórica de estimados pasados.

C. Elaborar una primera estimación base del proyecto de software

De tal forma, ya se sabe qué se va a estimar, ahora se debe llevar el foco al “¿Cómo?”. Para esto existen una amplia gama de técnicas de estimación, por ejemplo:

- **Métodos de medición de Proyectos:** Estimación de 3 puntos, Técnicas Delphi, COCOMO, Putam, y actualmente, Planning Poker (para proyectos con metodología ágil).
- **Métodos de medición de Puntos Función:** FPUG FPA, MKII-FPA, COSMIC FFP, FiSMA, Estimación temprana de NESMA, SiFP o Puntos Función 3D.
- **Otros métodos de medición funcional:** Puntos de características (Feature Points), Puntos de Casos de Uso o Puntos Objeto.
- **Métodos de medición no funcional:** Lógica Difusa (Fuzzy Logic), Catalogo de Componentes (Standard Components), Puntos de Historia, T-SHIRT Sizing o SNAP.

Una de las prácticas de medición de proyectos clásicos más usada, es la estimación de 3 puntos, que consiste en tomar el desglose de trabajo y establecer una estimación optimista, otra moderada y otra pesimista realizadas por un experto. Otra de las técnicas

clásicas es la Delphi, que consiste en enviar la información a 3 expertos independientes y ponderar sus estimaciones.

Las técnicas clásicas dependen del conocimiento del experto, mientras que las técnicas de puntos función o medición funcional, se basan más en metodologías que evalúan con precisión cada componente, ponderando la complejidad de cada uno, e introduciendo modificadores según lo estable del sistema o experiencia del equipo.

D.- Identificar y analizar los riesgos

- Una vez lograda la primera estimación base, ahora se debe considerar los riesgos.
- El riesgo no constituye una amenaza si es gestionado adecuadamente.
- El motivo deriva de la incertidumbre existente en cada componente a desarrollar. Por ejemplo:
 - Si una actividad depende de una condición, existe el riesgo que esta no se cumpla y esto produzca un impacto.
 - Todas las actividades tienen insumos (un producto previo) y recursos, si estos no están disponibles, provocan un impacto. Ej. Recursos escasos, personal no disponible o costoso, proyectos externos, etc.
- Todo esto provoca pérdida de tiempo, costo, calidad. A esta pérdida se le conoce como el impacto del riesgo.
- Para dar respuesta al riesgo se establecen acciones y planes de respuesta, tales como:
 - Realizar provisiones adicionales en las estimaciones parciales (agregar holgura).
 - Agregar más actividades y por tanto, mayor es la estimación, para contemplar el trabajo necesario para mitigar el riesgo.
- Una vez identificados los riesgos, se debe volver al paso 2 y 3 para revisar nuevamente nuestras estimaciones.

E.- Validación y revisión de la estimación

- Por último, siempre es una buena práctica realizar una evaluación formal de las estimaciones.
- Para que sea la revisión sea objetiva, se debe realizar por alguien que no participó en la elaboración de la estimación.
- En la evaluación, se debe revisar:
 - Las bases, el desglose de tareas, las premisas y las restricciones.
 - Si el procedimiento de estimación cumplió la metodología establecida.
- Debe enfocarse en identificar:
 - Premisas no ciertas o con alta probabilidad de no cumplirse.
 - Riesgos no identificados.
 - Requerimientos ocultos no identificados.
 - Posibles requerimientos extraordinarios.

Una vez obtenida la estimación,

- Se obtiene el número de horas de trabajo o jornadas asignadas a cada componente, la agregación de estos componentes da una estimación de esfuerzo del proyecto.
- Esto no significa que se tenga ya el plan de proyecto y el presupuesto, para ello, debo seguir una metodología de elaboración de cronograma y de presupuesto.
- Sin embargo, si se tiene una medición de la dimensión, se puede estimar un costo, multiplicando las jornadas (u horas) por la tarifa de los recursos comprometidos.

Ejemplos de estimación de costos de un proyecto de software

Cuando surja la necesidad de estimar cuánto dinero se necesita para desarrollar el próximo proyecto de software, es conveniente que se cuente con ejemplos de estimación de costos de proyectos de software anteriores, los cuales se puedan usar como referencia y guía.

Para realizar una estimación de costos de un proyecto de software se necesitarán dos cosas, en primer lugar determinar el tamaño del software que se va a desarrollar, utilizando alguna unidad de medida, luego, se necesitará saber cuántas unidades de dicha medida puede desarrollar el equipo de trabajo, a un determinado costo.

Método de medición del tamaño del software a desarrollar

Para determinar la estimación del software a desarrollar, primero debemos seleccionar un método para medir su tamaño. Para el ejemplo de estimación de costos de un proyecto de software utilizaremos el análisis de puntos de función por medio del método COSMIC.

COSMIC es un método de análisis de puntos de función de segunda generación, en el cual se determina el tamaño funcional del software a partir del número de interacciones entre los procesos funcionales.

Los pasos para realizar esta medición son los siguientes:

- Fase 1: Estrategia de medición.
- Fase 2: Mapeo.
- Fase 3: Medición.

COSMIC tiene la ventaja de no establecer límites arbitrarios al tamaño funcional, así puede medir componentes de software muy grandes o pequeños. Adicionalmente, está basado en el desglose funcional de los componentes de software, alineado con las prácticas de Ingeniería de software.

1.2.3 Ejemplo para medir el software o componente a desarrollar

Estrategia de medición

Se determina identificando cuales son los requerimientos funcionales a medir, cual es el propósito de la medición y quienes son los usuarios funcionales.

Requerimientos funcionales del proyecto:

Pedidos pendientes de facturar

La facturación de pedidos de venta se realizara en lotes, a través de una pantalla de pedidos pendientes de facturación, la cual debe mostrar los pedidos no facturados. Estos pedidos se podrán consultar por Fecha de inicio y fecha fin, o por cada cliente. Esta lista de pedidos pendientes de venta se podrá exportar o imprimir a un archivo Excel.

Facturación de pedidos

Desde la pantalla de pedidos pendientes, el usuario podrá seleccionar uno o varios pedidos para ser facturados. Se deberá poder facturar un pedido individual o un lote de pedidos. El usuario podrá elegir entre emitir una factura por pedido o agrupar las facturas por cliente. Una vez facturado, se debe cambia el estatus del pedido a “facturado”, y así, al refrescar la lista de pedidos pendientes este no será mostrado.

Finalidad de la medición

Realizar la estimación funcional de mejora del sistema de administración, que abarca los requisitos de lista de pedidos pendientes, la facturación de pedidos y el envío de email al facturar el pedido.

Usuarios

El usuario principal funcional del sistema es el Analista del departamento de facturación y cuentas por cobrar.

Mapeo y medición

En el método COSMIC, se utiliza ingeniería de software del proyecto para determinar los procesos funcionales y los flujos de datos que lo componen. Posteriormente, se asigna un punto de función COSMIC por cada movimiento de datos identificado.

El ejemplo está compuesto por los siguientes procesos funcionales y movimientos de datos:

Proceso funcional: Visualizar lista de pedidos pendientes de facturación.

Flujos de datos:

- Entrada: Seleccionar en el menú la opción “lista de pedidos pendientes”.
- Lectura: Obtener todos los pedidos pendientes de venta.
- Salida: Sacar en pantalla todos los pedidos de venta pendientes.
- Entrada: Especificar fecha inicial y final del pedido.
- Entrada: Especificar cliente de pedido.
- Lectura: Obtener pedidos de venta filtrados a partir de los parámetros de búsqueda.
- Salida: Mostrar en pantalla la lista de pedidos pendientes según filtrado de criterios.
- Salida: Imprimir la lista de pedidos.
- Salida: Exportar la lista de pedidos a Excel.

Puntos de función COSMIC: 9 CFP.

Proceso funcional: Facturar pedidos pendientes.

Movimientos de datos:

- Entrada: Seleccionar el pedido a facturar.
- Entrada: Seleccionar la facturación individual o agrupada por cliente por cada pedido.
- Entrada: Iniciar proceso de facturación actuando sobre un botón de inicio.
- Lectura: Leer de la base de datos los datos de pedidos seleccionados para facturación.
- Lectura: Leer las líneas de cada pedido.
- Escritura: Crear un registro de factura para cada pedido (Facturación individual).
- Escritura: Crear registro de líneas de factura individual.
- Escritura: Agrupar pedidos de un mismo cliente y crear un registro de factura (Facturación por cliente).
- Escritura: Crear registro de líneas de factura por cliente.
- Escritura: Cambiar el estado de pedido a "facturado".

Puntos de función COSMIC: 10 CFP.

De esta forma, se ha determinado que el proyecto tiene una medición de:

19 puntos de función COSMIC (19 CFP).

Costo del equipo de trabajo de desarrollo de software

Para poder determinar el costo de desarrollo del software, necesitamos valernos de la información de proyectos pasados que tenga la organización. También podemos usar información de otras fuentes, otras organizaciones y bases de datos de Benchmark.

Supongamos que tenemos un equipo de desarrollo de software y se sabe que su costo mensual es de 21.800 euros.

Para determinar este costo, se debe considerar el número de personas, cual es la remuneración de cada rol, sí, por ejemplo: desarrollador, analista de prueba, diseñador, jefe de proyecto, etc. También, se debe considerar otros gastos del personal como las pagas extras, seguros, y también el costo administrativo de cada persona, infraestructura donde trabaja, gastos de gerencia y administración, entre otros.

Si este proyecto se realiza para un tercero, se debe añadir además el margen de ganancia (beneficio industrial) que se espera obtener.

Unidades de medida capaces de desarrollar el equipo de trabajo en un tiempo determinado

Ahora, supongamos que, examinando la información histórica de la organización, se puede determinar que, en los últimos 12 meses, el equipo de trabajo ha producido un promedio de 23 puntos de función COSMIC mensuales.

Para que esta medición sea exacta, se considera que un punto de función está desarrollado solamente cuando está totalmente acabado e instalado en el entorno de producción. Si una

funcionalidad se ha desarrollado, pero aún está a falta de pruebas o aún no ha pasado al entorno de producción, no debe contar para el cálculo del promedio de los puntos de función desarrollados en el mes.

Determinar el costo por unidad de medida

Para determinar el costo de cada punto de función se utiliza la siguiente formula:

$$\text{Costo punto de función} = \text{Costo mes del equipo de trabajo} / \text{puntos de función del mes}$$

De nuestro ejemplo:

$$\text{Costo por punto de función} = 21.800 \text{ €} / 23 \text{ puntos de función} = 947,82 \text{ €} / \text{Punto de función}$$

Una vez que se cuenta con la medición del tamaño del software y el costo por unidad de medida, se puede determinar el costo del proyecto de software usando la siguiente formula:

$$\text{Costo de un proyecto de software} = \text{Tamaño del software} \times \text{Costo por punto de función}$$

En nuestro ejemplo de estimación de costos de un proyecto de software lo determinamos de la siguiente forma:

$$\text{Costo del proyecto de software} = 19 \text{ CFP} \times 947,82 \text{ €}$$

$$\text{Costo del proyecto de software} = 18.008,69 \text{ €}$$

Tiempo que durará el proyecto de desarrollo de software

Los puntos de función COSMIC los podemos utilizar también para determinar cuánto tiempo durará el proyecto de software.

En el ejemplo, se sabe que el equipo de desarrollo de software produce 23 puntos de función al mes y se sabe también que el software que se va a desarrollar está estimado en x puntos de función. Si se divide el tamaño funcional del software entre el número de puntos de función mes se podrá determinar el número de meses que durará el proyecto.

$$\text{Duración del proyecto} = 19 \text{ puntos de función COSMIC} / 23 \text{ puntos de función COSMIC mes}$$

$$\text{Duración del proyecto} = 0,83 \text{ meses}$$

Conclusión:

De esta forma se ha determinado que el proyecto de software durará 0,83 meses en desarrollarse (Poco menos de un mes) y costará 18.008,69 €

1.2.4 Medición de atributos internos del producto. Longitud

Los principales productos que es útil medir son: la especificación, el diseño y el código.

El número de líneas de código (LOC) es la medida más usada para medir la longitud del código fuente. Se han realizado muchas propuestas para su contabilización. Una de ellas:

Líneas de código efectivas (ELOC: effective lines of code) o no comentadas (NCLOC)

- Es la más extendida

- No contabiliza las líneas comentadas ni en blanco

Es útil medir por separado las líneas comentadas (CLOC) de las no comentadas para calcular esfuerzo, productividad, etc. La longitud total será:

$$LOC = NCLOC + CLOC$$

También puede ser útil calcular la densidad de comentarios:

$$CLOC/LOC$$

Para propósitos tales como la prueba es importante conocer cuanto código ejecutable se produce, para ello se mide el número de sentencias ejecutables (ES), ignorando los comentarios, declaraciones de datos y cabeceras

Otra propuesta consiste en contabilizar únicamente el código entregado al cliente. Se cuenta el número de DSI (delivered source instructions) que incluye las declaraciones de datos, las cabeceras y las instrucciones fuente.

1.2.4.1 Métricas de Halstead [Halstead, 1977]

Considera un programa P como un conjunto de tokens (token: unidad sintáctica más elemental distinguible por un compilador) que se pueden clasificar como operandos y operadores

- Las métricas básicas para los tokens son:
 - n1: número de operadores únicos
 - n2: número de operandos únicos
 - N1: número total de ocurrencias de operadores
 - N2: número total de ocurrencias de operandos

Las métricas compuestas para un programa son:

- Longitud estimada:

$$L = n1 \log_2 n1 + n2 \log_2 n2$$

- Volumen:

$$V = N \log_2 n$$

$$\text{donde } N = N1 + N2 \text{ y } n = n1 + n2$$

- Nivel:

$$L = V^*/V$$

donde V^* = volumen del tamaño mínimo

- Esfuerzo de implementación:

$$E = (n1 N2 N \log_2 n) / (2 n2)$$

1.2.4.2 Especificaciones y diseño

Los documentos de especificación de requisitos y de diseño tienen representaciones de muchos tipos (texto, gráficos, símbolos...). La medición del atributo longitud exige la identificación de elementos atómicos que puedan contarse. Ejemplo:

- Diagramas de flujo de datos: procesos, entidades externas, almacenes de datos y flujo de datos
- Especificaciones algebraicas: salidas, funciones, operaciones y axiomas
- Esquemas Z: líneas de especificación (declaraciones y predicados)

Se pueden definir páginas de documentación como objetos atómicos

Vista	Diagrama	Objetos atómicos
Funcional	Diagrama de flujo de datos Diccionario de datos	Burbujas Elementos de datos
Datos	Diagrama entidad relación	Objetos, relaciones
Estado	Diagrama de transición de estados	Estados, transiciones

Tabla 2.1 Componentes atómicos del análisis estructurado

1.2.4.3 Medición de atributos internos del producto. Funcionalidad

Para realizar estimaciones de esfuerzo y duración es mejor estimar el tamaño del producto en base a su funcionalidad. La mayoría de los enfoques miden la funcionalidad de los documentos de especificación

Puntos de función

- El análisis de los puntos de función (FPA) es una medida de la funcionalidad propuesta en el método de estimación del esfuerzo de Albrecht [Albrecht, 1979]
- El paso previo al cálculo de los puntos de función, FP, es el cálculo de UFC (unadjusted function point count):

Se determinan los siguientes elementos de alguna representación del software:

- **Entradas externas:** entradas de usuario que proporcionan datos a la aplicación
- **Salidas externas:** Salidas que proporcionan información al usuario
- **Consultas externas:** peticiones interactivas que requieren una respuesta
- **Ficheros externos:** interfaces con otros sistemas legibles por la máquina
- **Ficheros internos:** ficheros maestros lógicos del sistema

A cada ítem se le asigna un índice de complejidad (simple, medio o complejo) y un factor de peso en función del índice, cuyos valores se recogen en la tabla siguiente

Factor de peso

Item	Simple	Medio	Complejo
Entradas externas	3	4	6
Salidas externas	4	5	7
Consultas externas	3	4	6
Ficheros externos	7	10	15
Ficheros internos	5	7	10

Tabla 2.2 Factores de peso en función del índice de complejidad.

UFC se calcula mediante la siguiente fórmula:

$$UFC = \sum_{i=1}^{15} ((\text{número de ítems de la clase } i) * \text{peso } i)$$

Para completar el cálculo de los PF es necesario conocer el factor de complejidad técnica (TCF) que engloba los 14 factores que aparecen en la tabla siguiente:

Componentes del factor de complejidad técnica		
F1 Copias de seguridad y recuperación Reusabilidad	F6 Entrada interactiva de datos	F11 Reusabilidad
F2 Comunicación de datos	F7 Facilidad operativa	F12 Facilidad de instalación
F3 Funciones distribuidas	F8 Actualización interactiva	F13 Múltiples sitios
F4 Rendimiento	F9 Interfaces complejas	F14 Facilidad de cambios
F5 Configuración muy usada	F10 Procesamiento complejo	

Tabla 2.3. 14 componentes de la complejidad técnica.

Cada componente de la tabla anterior se sitúa en una escala entre 0 y 5 según su influencia:

Ninguna influencia	0	Medio	3
Incidental	1	Significativo	4
Moderado	2	Esencial	5

La siguiente fórmula combina los 14 factores:

$$TCF = 0.65 + 0.01 \sum_{i=1}^{14} F_i$$

Los valores constantes de la ecuación y los factores de ponderación se obtienen empíricamente. Cálculo final de los puntos de función:

$$FP = UFC * TCF$$

La técnica de puntos de función presenta muchos problemas debidos fundamentalmente a la subjetividad en la aplicación de los factores y a la inexactitud de las medidas.

1.2.4.4 Modificaciones y alternativas a FPA

Las críticas que presenta el método de Albrecht son: semántica confusa, método de contabilización complejo, y asignación arbitraria de pesos, sólo considerando funciones del usuario final, sin tener en cuenta el esfuerzo de implementación de funciones internas. Entre las alternativas que plantearon resolver esta problemática, están:

- Puntos de función Mark II (MkII FPA) [Symons, 1991]
- Medición del tamaño funcional [ISO/IEC 14143, 2003]
- IFPUG (The International Function Point Users' Group) [ISO/IEC 20926:2003]: estándar de uso de FPA
- NESMA [ISO/IEC 24570, 2003]
- COSMIC-FFP [ISO/IEC 19761:2003]
- UCP (Use Case Points) [Karner, 1993]

1.2.4.5 Medición de atributos internos del producto

Medición del tamaño funcional

- Conjunto de estándares [ISO/IEC 14143 parts 1 – 5] creados como respuesta a la gran proliferación de propuestas de PF
- Definiciones (parte 1):
 - **Tamaño funcional:** tamaño del software derivado de la cuantificación de los requisitos funcionales de usuario
 - **Medición del tamaño funcional:** proceso de medir el tamaño funcional
 - **Índices de productividad:** tamaño del producto software entregado / esfuerzo
 - **Efectividad del coste:** tamaño del producto software entregado / coste
 - **Calidad del producto:** defectos entregados / tamaño del producto software entregado

1.2.5 COSMIC-FFP [ISO/IEC 19761:2003]

Propuesta del COmmon Software Measurement International Consortium para sistemas cliente servidor y aplicaciones multicapa. Compatible con técnicas actuales de especificación de requisitos (*casos de uso y prototipado*). *Aplicable en proyectos que usan COTS.*

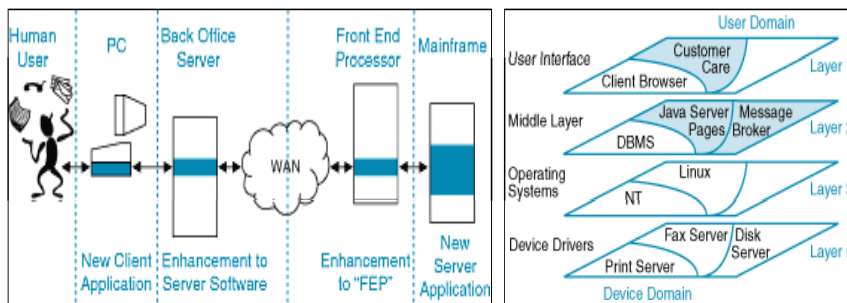


Figura 1.12 Ejemplo de un sistema cliente-servidor arquitectura software

Capas de la

1.2.6 UCP (Use Case Points)

Métrica similar al análisis de puntos de función, pero basada en la funcionalidad representada en forma de casos de uso.

- Los UCP se utilizan para estimar el esfuerzo de desarrollo⁷
- Considera actores, escenarios y factores técnicos y de entorno.

⁷ [Puntos de caso de uso - Wikipedia, la enciclopedia libre](#)

- Requiere el cálculo de tres variables:
 - UUCP (Unadjusted Use Case Points)
 - UUCW (Unadjusted Use Case Weight): considera el número y complejidad de los casos de uso.
 - UAW (Unadjusted Actor Weight): considera el número y complejidad de los actores.
 - TCF (Technical Complexity Factor).
 - ECF (Environment Complexity Factor).

Los UCP se calculan mediante la expresión:

$$UCP = UUCP * TCF * ECF$$

Se definen trece factores de complejidad técnica (TCF) y ocho factores de complejidad del entorno (ECF). A cada factor se le asigna un peso (W) acorde a su impacto y una complejidad percibida (F) correspondiente a la percepción de complejidad que tiene el equipo de desarrollo:

Cálculo de TCF:

$$TCF = C1 + C2 \sum_{j=1}^{13} W_i F_i$$

Cálculo de ECF:

$$ECF = C1 + C2 \sum_{j=1}^8 W_i F_i$$

Factores de complejidad técnica		
T1 Sistemas distribuidos	T6 Facilidad de instalación	T11 Características especiales de seguridad
T2 Rendimiento	T7 Facilidad de uso	T12 Acceso directo a terceras partes
T3 Eficiencia del usuario final	T8 Portabilidad	T13 Se requiere entrenamiento especial del usuario
T4 Procesamiento interno complejo	T9 Facilidad de cambio	
T5 Reusabilidad	T10 Concurrencia	

Tabla 2.4 Factores de complejidad técnica

Factores de complejidad del entorno	
E1 Familiaridad con UML	E5 Experiencia en orientación a objetos
E2 Trabajadores a tiempo parcial	E6 Motivación
E3 Capacidad de los analistas	E7 Dificultad del lenguaje de programación
E4 Experiencia en la aplicación	E8 Estabilidad de los requisitos

Tabla 2.5 Factores de complejidad del entorno.

1.2.7 Enfoque COCOMO

COCOMO es un modelo para predecir el esfuerzo en función del tamaño del sistema. Utiliza una medida del tamaño que puede ser aplicada al comienzo del desarrollo: **los puntos objeto**. El cálculo de los puntos objeto se realiza contabilizando el número de pantallas, informes y componentes de 3GL de la aplicación. A cada objeto se le asigna un factor de peso según su grado de dificultad

Tipo de objeto	Simple	Medio	Difícil
Pantalla	1	2	3
Informe	2	5	8
Componente 3GL	-	-	10

Tabla 2.6 Asignación de pesos a objetos en 3 niveles.

Los pesos reflejan el esfuerzo relativo requerido para implementar un objeto de un determinado nivel. Si existe reutilización los objetos ponderados se suman y se multiplican por un factor de reutilización para dar un nuevo número de puntos objeto:

$$\text{Puntos objeto nuevos} = (\text{puntos objeto}) * (100 - r) / 100$$

1.2.8 Métricas bang

Medida de la funcionalidad propuesta por DeMarco [DeMarco, 1978, 1982]. Para calcular la métrica se deben evaluar un conjunto de primitivas:

- Primitivas funcionales (PFu)
- Elementos de datos (ED)
- Objetos (OB)
- Relaciones (RE)
- Estados (ES)
- Transiciones (TR)

Se determinan cuentas adicionales para:

- Primitivas modificadas de la función manual (PMFu)
- Elementos de datos de entrada (EDE)
- Elementos de datos de salida (EDS)
- Elementos de datos retenidos (EDR)
- Muestras (tokens) de datos (TCi)
- Conexiones de relación (REi)

Se calculan para dos dominios:

- Dominio de la función: medida del número de primitivas funcionales de los diagramas de flujo
- Dominio de los datos: medida basada en el número de entidades y relaciones del diagrama entidad-relación

$$RE/PFu < 0,7 \quad \text{aplicación de dominio de función}$$

$0,8 < RE / PFu < 1,4$ aplicación híbrida
 $RE/PFu > 1,5$ aplicación de dominio de datos

Las métricas se calculan a partir de incrementos PFu y OB corregidos (IPFuC e IOBC):

TCi	IPFuC
2	1,0
5	5,8
10	16,6
15	29,3
20	43,2

REi	IOBC
1	1,0
3	4,0
6	9,8

Las métricas bang pueden definirse formalmente y calcularse automáticamente mediante herramientas CASE

1.2.9 Complejidad

Complejidad de un problema: cantidad de recursos que se requieren para una óptima solución del problema

Complejidad de la solución: recursos necesarios para implementar una solución particular:

- Complejidad de tiempo: el recurso es tiempo de ordenador
- Complejidad de espacio: el recurso es memoria de ordenador

La complejidad de la solución se puede medir en términos de eficiencia de esa solución, como puede ser la eficiencia algorítmica (función del número de operaciones primitivas que se requieren para una entrada dada):

Las entradas se pueden caracterizar mediante un parámetro de tamaño, n

Si F es el conjunto de todas las funciones de n , se puede establecer una relación ">" que se corresponda con la relación empírica "más eficiente"

La notación "big-O" permite definir una relación de orden sobre funciones $f(n)$ encontrando el término dominante e ignorando las constantes que multiplican: $O(g)$ es el conjunto de funciones que dominan asintóticamente la función g

La eficiencia de un algoritmo A es $O(f(n))$ si para una entrada de tamaño n se requieren $O(f(n))$ operaciones en el peor caso

Big-O de $f(n)$	Tipo de complejidad
$O(1)$	Complejidad constante
$O(\log n)$	Complejidad logarítmica
$O(n)$	Complejidad lineal
$O(n^2)$	Complejidad cuadrática
$O(n^i)$	Complejidad polinomial
$O(c^n), n > 1$	Complejidad exponencial

Tabla 2.7 Niveles de complejidad

La relación empírica "más eficiente" se representa mediante la relación de inclusión:

$$O(n) \sqsubset O(n^2)$$

La complejidad del problema se puede medir en función de la complejidad de sus soluciones utilizando la notación "big-O":

Un problema que tiene una solución de complejidad polinomial se dice que es factible o viable. La clase de todos los problemas factibles se llama P. Cualquier solución factible tiende a P. Hay problemas que no tienen una viabilidad conocida, pero existen métodos (algoritmos con límite polinomial) que se pueden aplicar para comprobar una solución propuesta. Esta clase de problemas se llaman NP. Los problemas NP tienen un subconjunto de problemas más complejos de categoría NP-total.

1.3 Complejidad Algorítmica

1.3.1 Eficiencia y complejidad

Respecto al uso eficiente de los recursos, éste suele medirse en función de dos parámetros: el espacio, es decir, memoria que utiliza, y el tiempo, lo que tarda en ejecutarse. Ambos representan los costes que supone encontrar la solución al problema planteado mediante un algoritmo. Dichos parámetros van a servir además para comparar algoritmos entre sí, permitiendo determinar el más adecuado de entre varios que solucionan un mismo problema. En este capítulo nos centraremos solamente en la eficiencia temporal.

El tiempo de ejecución de un algoritmo va a depender de diversos factores como son: los datos de entrada que le suministremos, la calidad del código generado por el compilador para crear el programa objeto, la naturaleza y rapidez de las instrucciones máquina del procesador concreto que ejecute el programa, y la complejidad intrínseca del algoritmo. Hay dos estudios posibles sobre el tiempo:

Uno que proporciona una medida teórica (a priori), que consiste en obtener una función que acote (por arriba o por abajo) el tiempo de ejecución del algoritmo para unos valores de entrada dados.

Y otro que ofrece una medida real (a posteriori), consistente en medir el tiempo de ejecución del algoritmo para unos valores de entrada dados y en un ordenador concreto.

Entendemos por tamaño de la entrada el número de componentes sobre los que se va a ejecutar el algoritmo. Por ejemplo, la dimensión del vector a ordenar o el tamaño de las matrices a multiplicar. La unidad de tiempo a la que debe hacer referencia estas medidas de eficiencia no puede ser expresada en segundos o en otra unidad de tiempo concreta, pues no existe un ordenador estándar al que puedan hacer referencia todas las medidas. Denotaremos por $T(n)$ el tiempo de ejecución de un algoritmo para una entrada de tamaño n .

Teóricamente $T(n)$ debe indicar el número de instrucciones ejecutadas por un ordenador idealizado. Debemos buscar por tanto medidas simples y abstractas, independientes del ordenador a utilizar. Para ello es necesario acotar de alguna forma la diferencia que se puede producir entre distintas implementaciones de un mismo algoritmo, ya sea del mismo código ejecutado por dos máquinas de distinta velocidad, como de dos códigos que implementen el mismo método. Esta diferencia es la que acota el siguiente principio:

Principio de Invarianza: Dado un algoritmo y dos implementaciones suyas I_1 e I_2 , que tardan $T_1(n)$ y $T_2(n)$ segundos respectivamente, el Principio de Invarianza afirma que existe una constante real $c > 0$ y un número natural n_0 tales que para todo $n \geq n_0$, se verifica que: $T_1(n) \leq cT_2(n)$.

Es decir, el tiempo de ejecución de dos implementaciones distintas de un algoritmo dado no va a diferir más que en una constante multiplicativa.

Con esto podemos definir sin problemas que un algoritmo tarda un tiempo del orden de $T(n)$ si existen una constante real $c > 0$ y una implementación I del algoritmo que tarda menos que $cT(n)$, para todo n tamaño de la entrada. Dos factores para tener muy en cuenta son la constante multiplicativa y el n_0 para los que se verifican las condiciones, pues si bien a priori un algoritmo de orden cuadrático es mejor que uno de orden cúbico, en el caso de tener dos algoritmos cuyos tiempos de ejecución son $10^6 n^2$ y $5n^3$ el primero sólo será mejor que el segundo para tamaños de la entrada superiores a 200.000.

También es importante hacer notar que el comportamiento de un algoritmo puede cambiar notablemente para diferentes entradas (por ejemplo, lo ordenados que se encuentren ya los datos a ordenar). De hecho, para muchos programas el tiempo de ejecución es en realidad una función de la entrada específica, y no sólo del tamaño de ésta. Así suelen estudiarse tres casos para un mismo algoritmo: caso peor, caso mejor y caso medio.

El caso mejor corresponde a la traza (secuencia de sentencias) del algoritmo que realiza menos instrucciones. Análogamente, el caso peor corresponde a la traza del algoritmo que realiza más instrucciones. Respecto al caso medio, corresponde a la traza del algoritmo que realiza un número de instrucciones igual a la esperanza matemática de la variable aleatoria definida por todas las posibles trazas del algoritmo para un tamaño de la entrada dado, con las probabilidades de que éstas ocurran para esa entrada.

A la hora de medir el tiempo, siempre lo haremos en función del número de operaciones elementales que realiza dicho algoritmo, entendiendo por operaciones elementales (en adelante OE) aquellas que el ordenador realiza en tiempo acotado por una constante. Así, consideraremos OE las operaciones aritméticas básicas, asignaciones a variables de tipo predefinido por el compilador, los saltos (llamadas a funciones y procedimientos, retorno desde ellos, etc.), las comparaciones lógicas y el acceso a estructuras indexadas básicas, como son los vectores y matrices. Cada una de ellas contabilizará como 1 OE.

En general, es posible realizar el estudio de la complejidad de un algoritmo sólo en base a un conjunto reducido de sentencias, aquellas que caracterizan que el algoritmo sea lento o rápido en el sentido que nos interesa. Para hacer un estudio del tiempo de ejecución de un algoritmo para los tres casos citados comenzaremos con un ejemplo concreto. Supongamos entonces que disponemos de la definición de los siguientes tipos y constantes:

CONST n = ...; (* num. máximo de elementos de un vector *); TYPE vector = ARRAY [1..n] OF INTEGER;

y de un algoritmo cuya implementación es:

```
PROCEDURE Buscar (VAR a:vector; c:INTEGER):CARDINAL;
VAR j:CARDINAL;
BEGIN
    J:=1; (* 1 *)
    WHILE (a[J]<c) AND (j<n) DO (* 2 *)
        J:=J+1 (* 3 *)
    END; (* 4 *)
    IF a[J]=c THEN (* 5 *)
        RETURN J (* 6 *)
    ELSE RETURN 0 (* 7 *)
    END (* 8 *)
END Buscar;
```

Para determinar el tiempo de ejecución, calcularemos primero el número de operaciones elementales (OE) que se realizan:

- En la línea (1) se ejecuta 1 OE (una asignación).
- En la línea (2) se efectúa la condición del bucle, con un total de 4 OE (dos comparaciones, un acceso al vector, y un AND).
- La línea (3) está compuesta por un incremento y una asignación (2 OE).
- La línea (5) está formada por una condición y un acceso al vector (2 OE).
- La línea (6) contiene un RETURN (1 OE) si la condición se cumple.
- La línea (7) contiene un RETURN (1 OE), cuando la condición del IF anterior es falsa.

No se contabiliza la copia del vector a la pila de ejecución del programa, pues se pasa por referencia y no por valor (está declarado como argumento VAR, aunque no se modifique dentro de la función). En caso de pasarlo por valor, necesitaríamos tener en cuenta el coste que esto supone (un incremento de n OE).

En el caso mejor para el algoritmo, se efectuará la línea (1) y de la línea (2) sólo la primera mitad de la condición, que supone 2 OE (suponemos que las expresiones se evalúan de izquierda a derecha, y con "cortocircuito", es decir, una expresión lógica deja de ser evaluada

en el momento que se conoce su valor, aunque no hayan sido evaluados todos sus términos). Tras ellas la función acaba ejecutando las líneas (5) a (7). En consecuencia, $T(n)=1+2+3=6$. • En el caso peor, se efectúa la línea (1), el bucle se repite $n-1$ veces hasta que se cumple la segunda condición, después se efectúa la condición de la línea (5) y la función acaba al ejecutarse la línea (7). Cada iteración del bucle está compuesta por las líneas (2) y (3), junto con una ejecución adicional de la línea (2) que es la que ocasiona la salida del bucle. Por tanto:

$$T(n) = 1 + \left\{ \left(\sum_{i=1}^{n-1} (4 + 2) \right) + 4 \right\} + 2 + 1 = 6n + 2$$

En el caso medio, el bucle se ejecutará un número de veces entre 0 y $n-1$, y vamos a suponer que cada una de ellas tiene la misma probabilidad de suceder. Como existen n posibilidades (puede que el número buscado no esté) suponemos a priori que son equiprobables y por tanto cada una tendrá una probabilidad asociada de $1/n$. Con esto, el número medio de veces que se efectuará el bucle es de:

$$\sum_{i=1}^{n-1} i \frac{1}{n} = \frac{n-1}{2}$$

Tenemos pues que:

$$T(n) = 1 + \left\{ \left(\sum_{i=1}^{(n-1)/2} (4 + 2) \right) + 2 \right\} + 2 + 1 = 3n + 3$$

Es importante observar que no es necesario conocer el propósito del algoritmo para analizar su tiempo de ejecución y determinar sus casos mejor, peor y medio, sino que basta con estudiar su código.

En este caso, un examen más detallado de la función nos muestra que tras su ejecución, la función devuelve la posición de un entero dado c dentro de un vector ordenado de enteros, devolviendo 0 si el elemento no está en el vector. Lo que acabamos de probar es que su caso mejor se da cuando el elemento está en la primera posición del vector. El caso peor se produce cuando el elemento no está en el vector, y el caso medio ocurre cuando consideramos equiprobables cada una de las posiciones en las que puede encontrarse el elemento dentro del vector (incluyendo la posición especial 0, que indica, como se ha comentado, que el elemento a buscar no se encuentra en el vector).

1.3.2 Reglas generales para el cálculo del número de OE

La siguiente lista presenta un conjunto de reglas generales para el cálculo del número de OE, siempre considerando el peor caso. Estas reglas definen el número de OE de cada estructura básica del lenguaje, por lo que el número de OE de un algoritmo puede hacerse por inducción sobre ellas.

- Vamos a considerar que el tiempo de una OE es, por definición, de orden 1. La constante c que menciona el Principio de Invarianza dependerá de la implementación particular, pero nosotros supondremos que vale 1.
- El tiempo de ejecución de una secuencia consecutiva de instrucciones se calcula sumando los tiempos de ejecución de cada una de las instrucciones.
- El tiempo de ejecución de la sentencia "CASE C OF $v_1:S_1 \mid v_2:S_2 \mid \dots \mid v_n:S_n$ END;" es :
 $T = T(C) + \max\{T(S_1), T(S_2), \dots, T(S_n)\}$. Obsérvese que $T(C)$ incluye el tiempo de comparación con $v_1, v_2, \dots, v_n$.
- El tiempo de ejecución de la sentencia "IF C THEN S_1 ELSE S_2 END;" es
 $T = T(C) + \max\{T(S_1), T(S_2)\}$.
- El tiempo de ejecución de un bucle de sentencias "WHILE C DO S END;" es

$$T = T(C) + (n^{\circ} \text{ iteraciones}) * (T(S) + T(C)).$$

- Obsérvese que tanto $T(C)$ como $T(S)$ pueden variar en cada iteración, y por tanto habrá que tenerlo en cuenta para su cálculo.
- Para calcular el tiempo de ejecución del resto de sentencias iterativas (FOR, REPEAT, LOOP) basta expresarlas como un bucle WHILE. A modo de ejemplo, el tiempo de ejecución del bucle:

```
FOR i:=1 TO n DO
    S
END;
```

puede ser calculado a partir del bucle equivalente:

```
i:=1;
WHILE i<=n DO
    S; INC(i)
END;
```

- El tiempo de ejecución de una llamada a un procedimiento o función $F(P_1, P_2, P_3, \dots, P_n)$ es 1 (por la llamada), más el tiempo de evaluación de los parámetros $P_1, P_2, P_3, \dots, P_n$, más el tiempo que tarde en ejecutarse F , esto es, $T = 1 + T(P_1) + T(P_2) + \dots + T(P_n) + T(F)$. No contabilizamos la copia de los argumentos a la pila de ejecución, salvo que se trate de estructuras complejas (registros o vectores) que se pasan por valor.

El paso de parámetros por referencia, por tratarse simplemente de punteros, no contabiliza tampoco.

También es necesario tener en cuenta, cuando el compilador las incorpore, las optimizaciones del código y la forma de evaluación de las expresiones, que pueden ocasionar "cortocircuitos" o realizarse de forma "perezosa" (lazy). En el presente trabajo supondremos que no se realizan optimizaciones, que existe el cortocircuito y que no existe evaluación perezosa.

1.3.3 Cotas de complejidad. Medidas asintóticas

Una vez vista la forma de calcular el tiempo de ejecución T de un algoritmo, nuestro propósito es intentar clasificar dichas funciones de forma que podamos compararlas. Para ello, y a modo visual de resumen de lo que viene a continuación, vamos a definir clases de equivalencia, correspondientes a las funciones que "crecen de la misma forma".

En las siguientes definiciones N denotará el conjunto de los números naturales y R el de los reales.

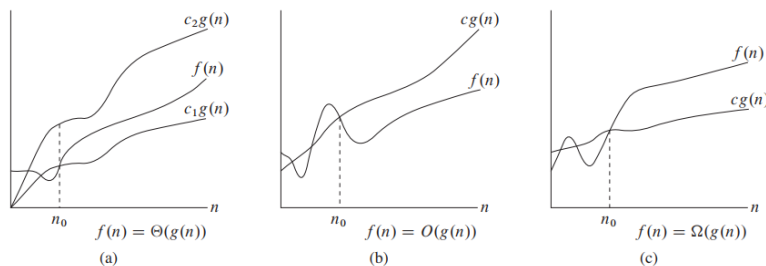


Figura 1.13 Notaciones de complejidad.

- (a) La θ -notación delimita una función entre factores constantes. Escribimos $f(n) = \theta(g(n))$. Si existen constantes positivas n_0 , c_1 y c_2 tales que a la derecha de n_0 , el valor de $f(n)$ siempre se encuentra entre $c_1g(n)$ y $c_2g(n)$ inclusive.
- (b) La notación-O da un **límite superior** para una función dentro de un factor constante. Escribimos $f(n) = O(g(n))$ si hay constantes positivas n_0 y c tales que en y a la derecha de n_0 , el valor de $f(n)$ siempre se encuentra en o por debajo de $cg(n)$.
- (c) La Ω -notación da un **límite inferior** para una función dentro de un factor constante. Escribimos $f(n) = \Omega(g(n))$ si hay constantes positivas n_0 y c tales que en y a la derecha de n_0 , el valor de $f(n)$ siempre se encuentra en o por encima de $cg(n)$.

1.3.3.1 Cota Superior. Notación O

Dada una función f , queremos estudiar aquellas funciones g que a lo sumo crecen tan deprisa como f . Al conjunto de tales funciones se le llama cota superior de f y lo denominamos $O(f)$. Conociendo la cota superior de un algoritmo podemos asegurar que, en ningún caso, el tiempo empleado será de un orden superior al de la cota.

Definición 1.1

Sea $f: N \rightarrow [0, \infty)$. Se define el conjunto de funciones de orden O (Ómicron) de f como:

$$O(f) = \{g: N \rightarrow [0, \infty) \mid \exists c \in \mathbb{R}, c > 0, \exists n_0 \in \mathbb{N} \cdot g(n) \leq cf(n) \forall n \geq n_0\}.$$

Diremos que una función $t: N \rightarrow [0, \infty)$ es de orden O de f si $t \in O(f)$.

Intuitivamente, $t \in O(f)$ indica que t está acotada superiormente por algún múltiplo de f . Normalmente estaremos interesados en la menor función f tal que t pertenezca a $O(f)$.

Usamos la notación O para dar un límite superior en una función, en un factor constante. La figura 1.13 (b) muestra la idea de la notación- O . Para todos los valores n en y a la derecha de n_0 , el valor de la función $f(n)$ es igual o inferior a $cg(n)$. Escribimos $f(n) = O(g(n))$ para indicar que una función $f(n)$ es miembro del conjunto $O(g(n))$. Tenga en cuenta que $f(n) = \Theta(g(n))$ implica $f(n) = O(g(n))$, ya que la notación- Θ tiene más fuerza que la notación- O . Escrito en teoría de conjuntos, tenemos $\Theta(g(n)) \subseteq O(g(n))$. Por tanto, nuestra prueba de que cualquier función cuadrática $an^2 + bn + c$, donde $a > 0$, está en $\Theta(n^2)$, también muestra que dicha función cuadrática está en $O(n^2)$. Lo que sorprende más es que cuando $a > 0$, cualquier función lineal $an + b$ está en $O(n^2)$, lo que se verifica fácilmente tomando $c = a + |b|$ y $n_0 = \max(1, -b/a)$.

Si se conoce la notación- O , puede que resulte extraño que debamos escribir, por ejemplo, $n = O(n^2)$. En la literatura, a veces encontramos notación- O que describe informalmente límites asintóticamente estrechos, es decir, lo que hemos definido usando la notación- Θ . Aquí, sin embargo, cuando escribimos $f(n) = O(g(n))$, simplemente estamos afirmando que algún múltiplo constante de $g(n)$ es un límite superior asintótico en $f(n)$, sin ninguna afirmación sobre qué cuán estrecho ese límite superior es. Distinguir los límites superiores asintóticos de los límites asintóticamente estrechos es un tema en sí en la bibliografía de algoritmos.

Usando la notación- O , a menudo podemos describir el tiempo de ejecución de un algoritmo simplemente inspeccionando la estructura general del algoritmo. Por ejemplo, la estructura de bucle doblemente anidado del algoritmo de 'ordenación por inserción', produce inmediatamente un límite superior $O(n^2)$ en el tiempo de ejecución del peor de los casos: el costo de cada iteración del bucle interno está acotado desde arriba por $O(1)$ (constante), los índices i y j son como máximo n , el bucle interno se ejecuta como máximo una vez por cada uno de los n^2 pares para i y j .

Por lo tanto, el límite de $O(n^2)$ en el peor de los casos en la ordenación por inserción, también se aplica a su tiempo de ejecución en cada entrada. Sin embargo, el límite $\Theta(n^2)$ en el peor caso en el caso de la ordenación por inserción no implica un límite, $\Theta(n^2)$ en tiempo de ejecución para cada entrada.

Para un n dado, el tiempo de ejecución real varía, dependiendo de la entrada particular de ese tamaño n . Cuando decimos "el tiempo de ejecución es $O(n^2)$ " queremos decir que hay una función $f(n)$ de $O(n^2)$ tal que, para cualquier valor de n , no importa qué entrada particular de tamaño n se elija, el tiempo de ejecución de esa entrada está limitado por arriba por el valor $f(n)$. De manera equivalente, queremos decir que el peor tiempo de ejecución es $O(n^2)$.

1.3.3.2 Propiedades de O

Veamos las propiedades de la cota superior. La demostración de todas ellas se obtiene aplicando la definición.

1. Para cualquier función f se tiene que si $f \in O(f)$:
2. $f \in O(g) \Rightarrow O(f) \subset O(g)$.
3. $O(f) = O(g) \Leftrightarrow f \in O(g)$ y $g \in O(f)$.
4. Si $f \in O(g)$ y $g \in O(h) \Rightarrow f \in O(h)$.
5. Si $f \in O(g)$ y $f \in O(h) \Rightarrow f \in O(\min(g, h))$.
6. Regla de la suma: Si $f_1 \in O(g)$ y $f_2 \in O(h) \Rightarrow f_1 + f_2 \in O(\max(g, h))$.
7. Regla del producto: Si $f_1 \in O(g)$ y $f_2 \in O(h) \Rightarrow f_1 \cdot f_2 \in O(g \cdot h)$.
8. Si existe

$\lim_{n \rightarrow \infty} \frac{f(n)}{f(n)} = k$, dependiendo de los valores que tome k obtenemos:

- a) Si $k \neq 0$ y $k < \infty$ entonces $O(f) = O(g)$.
- b) Si $k = 0$ entonces $f \in O(g)$, es decir, $O(f) \subset O(g)$, pero sin embargo se verifica que $g \notin O(f)$.

De las propiedades anteriores se deduce que la relación $\sim O$, definida por $f \sim O g$ si y sólo si $O(f) = O(g)$, es una relación de equivalencia. Siempre escogeremos el representante más sencillo para cada clase; así los órdenes de complejidad constante serán expresados por $O(1)$, los lineales por $O(n)$, etc.

1.3.3.3 Cota Inferior. Notación Ω

Dada una función f , queremos estudiar aquellas funciones g que a lo sumo crecen tan lentamente como f . Al conjunto de tales funciones se le llama cota inferior de f y lo denominamos $\Omega(f)$. Conociendo la cota inferior de un algoritmo podemos asegurar que, en ningún caso, el tiempo empleado será de un orden inferior al de la cota. Así como la notación- O proporciona un límite superior asintótico en una función, la notación- Ω proporciona un límite inferior asintótico. Para una función dada $g(n)$, denotamos por $\Omega(g(n))$ (pronunciado "omega grande de g de n " o, a veces, simplemente "omega de g de n ") el conjunto de funciones que:

Definición 1.2

Sea $f: \mathbb{N} \rightarrow [0, \infty)$. Se define el conjunto de funciones de orden Ω (Omega) de f como:

$$\Omega(f) = \{ g: \mathbb{N} \rightarrow [0, \infty) \mid \exists c \in \mathbb{R}, c > 0, \exists n_0 \in \mathbb{N} \bullet g(n) \geq cf(n) \forall n \geq n_0 \}.$$

Diremos que una función $t: \mathbb{N} \rightarrow [0, \infty)$ es de orden Ω de f si $t \in \Omega(f)$.

Intuitivamente, $t \in \Omega(f)$ indica que t está acotada inferiormente por algún múltiplo de f . Normalmente estaremos interesados en la mayor función f tal que t pertenezca a $\Omega(f)$, a la que denominaremos su cota inferior.

Obtener buenas cotas inferiores es en general muy difícil, aunque siempre existe una cota inferior trivial para cualquier algoritmo: al menos hay que leer los datos y luego escribirlos, de forma que ésa sería una primera cota inferior. Así, para ordenar n números una cota inferior sería n , y para multiplicar dos matrices de orden n sería n^2 ; sin embargo, los mejores algoritmos conocidos son de órdenes $n \log n$ y $n^{2.8}$ respectivamente.

La figura 1.13 (c) muestra la idea tras la notación- Ω . Para todos los valores n en o a la derecha de n_0 , el valor de $f(n)$ es igual o superior a $cg(n)$. A partir de las definiciones de las notaciones asintóticas que hemos visto hasta ahora, es fácil probar el siguiente teorema importante:

Teorema 3.1

Para dos funciones cualesquiera $f(n)$ y $g(n)$, tenemos $f(n) = \theta(g(n))$ si y solo si $f(n) = O(g(n))$ y $f(n) = \Omega(g(n))$.

Como ejemplo de la aplicación de este teorema, nuestra prueba de que $an^2 + bn + c = \theta(n^2)$, para cualesquiera constantes a, b y c , donde $a > 0$, implica inmediatamente que $an^2 + bn + c = \Omega(n^2)$, y $an^2 + bn + c = O(n^2)$. En la práctica, en lugar de usar el teorema 3.1 para obtener límites superior e inferior asintóticos a partir de límites asintóticamente estrechos, como hicimos en este ejemplo, generalmente lo usamos para demostrar límites asintóticamente estrechos a partir de límites superiores e inferiores asintóticos.

Cuando decimos que el tiempo de ejecución de un algoritmo es $\Omega(g(n))$ queremos decir que: no importa qué entrada particular de tamaño n se elija para cada valor de n , el tiempo de ejecución en esa entrada es al menos una constante multiplicada por $g(n)$, para n suficientemente grande. De manera equivalente, estamos dando un límite inferior en el mejor tiempo de ejecución de un algoritmo. Por ejemplo, el mejor tiempo de ejecución de la ordenación por inserción es $\Omega(n)$.

Por tanto, el tiempo de ejecución de la ordenación por inserción pertenece tanto $\Omega(n)$, como a $O(n^2)$, ya que cae en cualquier lugar entre una función lineal de n y una función cuadrática de n . Además, estos límites son asintóticamente lo más ajustados posible: a saber, el tiempo de ejecución de la ordenación por inserción no es $\Omega(n^2)$, ya que existe una entrada para la cual la ordenación por inserción se ejecuta en $\theta(n^2)$ tiempo (Cuando la entrada ya está ordenada).

Sin embargo, no es contradictorio decir que el peor tiempo de ejecución de la ordenación por inserción es $\Omega(n^2)$, ya que existe una entrada que hace que el algoritmo tome tiempo $\Omega(n^2)$.

1.3.3.4 Propiedades de Ω

Veamos las propiedades de la cota inferior Ω . La demostración de todas ellas se obtiene de forma simple aplicando la definición.

1. Para cualquier función f se tiene que $f \in \Omega(f)$.
2. $f \in \Omega(g) \Rightarrow \Omega(f) \subset \Omega(g)$.
3. $\Omega(f) = \Omega(g) \Leftrightarrow f \in \Omega(g)$ y $g \in \Omega(f)$.
4. Si $f \in \Omega(g)$ y $g \in \Omega(h) \Rightarrow f \in \Omega(h)$.
5. Si $f \in \Omega(g)$ y $f \in \Omega(h) \Rightarrow f \in \Omega(\max(g, h))$.
6. Regla de la suma: Si $f_1 \in \Omega(g)$ y $f_2 \in \Omega(h) \Rightarrow f_1 + f_2 \in \Omega(g + h)$.
7. Regla del producto: Si $f_1 \in \Omega(g)$ y $f_2 \in \Omega(h) \Rightarrow f_1 \cdot f_2 \in \Omega(g \cdot h)$.
8. Si existe $\lim_{n \rightarrow \infty} \frac{f(n)}{f(N)} = k$, dependiendo de los valores que tome k obtenemos:
 - a) Si $k \neq 0$ y $k < \infty$ entonces $\Omega(f) = \Omega(g)$.
 - b) Si $k = 0$ entonces $g \in \Omega(f)$, es decir, $\Omega(g) \subset \Omega(f)$, pero sin embargo se verifica que $f \notin \Omega(g)$.

De las propiedades anteriores se deduce que la relación $\sim\Omega$, definida por $f \sim\Omega g$ si y sólo si $\Omega(f) = \Omega(g)$, es una relación de equivalencia.

1.3.3.5 Orden Exacto. Notación Θ

Definición 1.3

Como última cota asintótica, definiremos los conjuntos de funciones que crecen asintóticamente de la misma forma.

Sea $f: \mathbb{N} \rightarrow [0, \infty)$. Se define el conjunto de funciones de orden Θ (Theta) de f como:

$$\Theta(f) = O(f) \cap \Omega(f)$$

o, lo que es igual:

$$\Theta(f) = \{g: \mathbb{N} \rightarrow [0, \infty) \mid \exists c, d \in \mathbb{R}, c, d > 0, \exists n_0 \in \mathbb{N} \cdot cf(n) \leq g(n) \leq df(n) \forall n \geq n_0\}.$$

Diremos que una función $t: \mathbb{N} \rightarrow [0, \infty)$ es de orden Θ de f si $t \in \Theta(f)$.

Intuitivamente, $t \in \Theta(f)$ indica que t está acotada tanto superior como inferiormente por múltiplos de f , es decir, que t y f crecen de la misma forma.

En su lugar, normalmente escribiremos " $f(n) = \Theta(g(n))$ " para expresar la misma noción. Puedes estar confundido porque abusamos de la igualdad de esta manera, pero veremos más adelante en esta sección que hacerlo tiene sus ventajas.

La figura 1.13 (a) ofrece una imagen intuitiva de las funciones $f(n)$ y $g(n)$. $f(n) = \Theta(g(n))$. Para todos los valores de n en y a la derecha de n_0 , el valor de $f(n)$ se encuentra en o por encima de $c_1 g(n)$ y en o por debajo de $c_2 g(n)$. En otras palabras, para todo $n \geq n_0$, la función $f(n)$ es homóloga a $g(n)$ dentro de un factor constante. Decimos que $g(n)$ es una cota asintóticamente ajustada para $f(n)$.

Existe una noción informal de notación- Θ que equivale a desechar los términos de orden inferior e ignorar el coeficiente principal del término de orden superior. Justifiquemos brevemente esta intuición utilizando la definición formal para demostrar que

$$\frac{1}{2}n^2 - 3n = \theta(n^2)$$

Para hacerlo, debemos determinar las constantes positivas c_1 , c_2 y n_0 tales que

$$c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$$

para todo $n \geq n_0$. Dividiendo entre n^2 se obtiene:

$$c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$$

Podemos hacer que la desigualdad de la derecha se mantenga para cualquier valor de $n \geq 1$ eligiendo cualquier constante ($c_2 \geq 1/2$). Del mismo modo, podemos hacer que la desigualdad de la izquierda se mantenga para cualquier valor de $n \geq 7$ eligiendo cualquier constante ($c_1 \geq 1/14$). Por lo tanto, al elegir ($c_1 = 1/14$), ($c_2 = 1/2$) y $n_0 = 7$, podemos verificar que:

$$\frac{1}{2}n^2 - 3n = \theta(n^2)$$

Existen también otras opciones para las constantes, pero lo importante es que existen opciones. Tenga en cuenta que estas constantes dependen de la función:

$$\frac{1}{2}n^2 - 3n$$

Una función diferente perteneciente a $\Theta(n^2)$ normalmente requeriría constantes diferentes.

También podemos usar la definición formal para verificar que $6n^3 \neq \theta(n^2)$. Suponga, a efectos de contradicción, que c_2 y n_0 existen de manera que $6n^3 \leq c_2 n^2$ para todo $n \geq n_0$. Dividiendo por n^2 produce $n \leq c_2 / 6$, que no puede ser válido para n arbitrariamente grande, ya que c_2 es constante.

Intuitivamente, los términos de orden inferior de una función asintóticamente positiva pueden ignorarse al determinar límites asintóticamente estrechos porque son insignificantes para n grandes. Cuando n es grande, incluso una pequeña fracción del término de orden superior es

suficiente para dominar los términos de orden inferior. Por tanto, establecer c_1 en un valor ligeramente menor que el coeficiente del término de orden más alto y establecer c_2 en un valor ligeramente mayor permite satisfacer las desigualdades en la definición de la notación- Θ . El coeficiente del término de orden superior también puede ignorarse, ya que solo cambia c_1 y c_2 por un factor constante igual al coeficiente.

Como ejemplo, considere cualquier función cuadrática $f(n) = an^2 + bn + c$, donde a , b y c son constantes y $a > 0$. Desechando los términos de orden inferior e ignorando los rendimientos constantes $f(n) = \Theta(n^2)$. Formalmente, para mostrar lo mismo, tomamos las constantes

$$(c_1 = a/4), (c_2 = 7a/4) \text{ y } n_0 = 2 \cdot \max\left(\frac{|b|}{a}, \sqrt{\frac{|c|}{a}}\right)$$

Puede verificar que $0 \leq c_1 n^2 \leq an^2 + bn + c \leq c_2 n^2$ para todo $n \geq n_0$. En general, para cualquier polinomio $p(n) = \sum_{i=0}^d a_i n^i$, donde a_i son constantes y $a_d > 0$, tenemos $p(n) = \Theta(n^d)$.

Dado que cualquier constante es un polinomio de grado 0, podemos expresar cualquier función constante como $\Theta(n^0)$, o $\Theta(1)$. Sin embargo, esta última notación es una "licencia menor", porque la expresión no indica qué variable tiende al infinito.⁸ A menudo usaremos la notación $\Theta(1)$ para significar una constante o una función constante con respecto a alguna variable.

Veamos las propiedades de la cota exacta. La demostración de todas ellas se obtiene también de forma simple aplicando la definición 1.3 y las propiedades de O y Ω .

1. Para cualquier función f se tiene que $f \in \Theta(f)$.
2. $f \in \Theta(g) \Rightarrow \Theta(f) = \Theta(g)$.
3. $\Theta(f) = \Theta(g) \Leftrightarrow f \in \Theta(g)$ y $g \in \Theta(f)$.
4. Si $f \in \Theta(g)$ y $g \in \Theta(h) \Rightarrow f \in \Theta(h)$.
5. Regla de la suma: Si $f_1 \in \Theta(g)$ y $f_2 \in \Theta(h) \Rightarrow f_1 + f_2 \in \Theta(\max(g, h))$.
6. Regla del producto: Si $f_1 \in \Theta(g)$ y $f_2 \in \Theta(h) \Rightarrow f_1 \cdot f_2 \in \Theta(g \cdot h)$.
7. Si existe $\lim_{n \rightarrow \infty} \frac{f(n)}{f(N)} = k$, dependiendo de los valores que tome k obtenemos:
 - a) Si $k \neq 0$ y $k < \infty$ entonces $\Theta(f) = \Theta(g)$.
 - b) Si $k = 0$ los órdenes exactos de f y g son distintos.

1.3.3.6 Observaciones sobre las cotas asintóticas

1. La utilización de las cotas asintóticas para comparar funciones de tiempo de ejecución se basa en la hipótesis de que son suficientes para decidir el mejor algoritmo, prescindiendo de las constantes de proporcionalidad. Sin embargo, esta hipótesis puede no ser cierta cuando el tamaño de la entrada es pequeño.
2. Para un algoritmo dado se pueden obtener tres funciones que miden su tiempo de ejecución, que corresponden a sus casos mejor, medio y peor, y que denominaremos respectivamente $T_m(n)$, $T_{1/2}(n)$ y $T_p(n)$. Para cada una de ellas podemos dar tres cotas asintóticas de crecimiento, por lo que se obtiene un total de nueve cotas para el algoritmo.
3. Para simplificar, dado un algoritmo diremos que su orden de complejidad es $O(f)$ si su tiempo de ejecución para el peor caso es de orden O de f , es decir, $T_p(n)$ es de orden $O(f)$. De forma análoga diremos que su orden de complejidad para el mejor caso es $\Omega(g)$ si su tiempo de ejecución para el mejor caso es de orden Ω de g , es decir, $T_m(n)$ es de orden $\Omega(g)$.

⁸ El problema real es que nuestra notación ordinaria para funciones no distingue funciones de valores. En λ -cálculo, los parámetros de una función están claramente especificados: la función n^2 podría escribirse como $\lambda n. n^2$, o incluso $\lambda r. r^2$. Sin embargo, adoptar una notación más rigurosa complicaría las manipulaciones algebraicas, por lo que optamos por tolerar el abuso.

4. Por último, diremos que un algoritmo es de orden exacto $\Theta(f)$ si su tiempo de ejecución en el caso medio $T_{1/2}(n)$ es de este orden.

1.3.4 Resolución de ecuaciones en recurrencia

Sin embargo, para los algoritmos recursivos nos vamos a encontrar con una dificultad añadida, pues la función que establece su tiempo de ejecución viene dada por una ecuación en recurrencia, es decir, $T(n) = E(n)$, en donde en la expresión E aparece la propia función T . Resolver tal tipo de ecuaciones consiste en encontrar una expresión no recursiva de T , y por lo general no es una labor fácil. Lo que veremos en esta sección es cómo se pueden resolver algunos tipos concretos de ecuaciones en recurrencia, que son las que se dan con más frecuencia al estudiar el tiempo de ejecución de los algoritmos desarrollados según las técnicas aquí presentadas.

1.3.4.1 Recurrencias homogéneas

Son de la forma:

$$a_0 T(n) + a_1 T(n-1) + a_2 T(n-2) + \dots + a_k T(n-k) = 0$$

donde los coeficientes a_i son números reales, y k es un número natural entre 1 y n . Para resolverlas vamos a buscar soluciones que sean combinaciones de funciones exponenciales de la forma:

$$T(n) = c_1 p_1(n) r_1^n + c_2 p_2(n) r_2^n + \dots + c_k p_k(n) r_k^n = \sum_{i=1}^k c_i p_i(n) r_i^n$$

donde los valores $c_1, c_2, \dots, c_n$, y $r_1, r_2, \dots, r_m$ son números reales, y $p_1(n), p_2(n), \dots, p_K(n)$, son polinomios en n con coeficientes reales.

Para resolverlas haremos el cambio $x^n = T(n)$, con lo cual obtenemos la ecuación característica asociada:

$$a_0 x^k + a_1 x^{k-1} + a_2 x^{k-2} + \dots + a_k = 0$$

Llamemos $r_1, r_2, \dots, r_k$ a sus raíces, ya sean reales o complejas. Dependiendo del orden de multiplicidad de tales raíces, pueden darse los dos siguientes casos.

Caso 1: Raíces distintas

Si todas las raíces de la ecuación característica son distintas, esto es, $r_i \neq r_j$ si $i \neq j$, entonces la solución de la ecuación en recurrencia viene dada por la expresión:

$$T(n) = c_1 r_1^n + c_2 r_2^n + c_3 r_3^n + \dots + c_n r_n^n = \sum_{i=1}^k c_i r_i^n$$

donde los coeficientes c_i se determinan a partir de las condiciones iniciales.

Como ejemplo, veamos lo que ocurre para la ecuación en recurrencia definida para la sucesión de Fibonacci:

$$T(n) = T(n-1) + T(n-2), n \geq 2$$

con las condiciones iniciales $T(0) = 0$, $T(1) = 1$. Haciendo el cambio $x^2 = T(n)$ obtenemos su ecuación característica $x^2 = x + 1$, o lo que es igual, $x^2 - x - 1 = 0$, cuyas raíces son:

$$r_1 = \frac{1 + \sqrt{5}}{2} \quad r_2 = \frac{1 - \sqrt{5}}{2}$$

y por tanto .

$$T(n) = c_1 \left\{ \frac{1 + \sqrt{5}}{2} \right\}^n + c_2 \left\{ \frac{1 - \sqrt{5}}{2} \right\}^n$$

Para calcular las constantes c_1 y c_2 necesitamos utilizar las condiciones iniciales de la ecuación original, obteniendo:

$$T(0) = c_1 \left\{ \frac{1+\sqrt{5}}{2} \right\}^0 + c_2 \left\{ \frac{1-\sqrt{5}}{2} \right\}^0 = c_1 + c_2 = 0$$



$$c_1 = -c_2 = \frac{1}{\sqrt{5}}$$

$$T(1) = c_1 \left\{ \frac{1+\sqrt{5}}{2} \right\}^1 + c_2 \left\{ \frac{1-\sqrt{5}}{2} \right\}^1 = 1$$

Sustituyendo entonces en la ecuación anterior, obtenemos

$$T(n) = \frac{1}{\sqrt{5}} \left\{ \frac{1+\sqrt{5}}{2} \right\}^n - \frac{1}{\sqrt{5}} \left\{ \frac{1-\sqrt{5}}{2} \right\}^n \in \mathcal{O}(\rho^n).$$

Caso 2: Raíces con multiplicidad mayor que 1

Supongamos que alguna de las raíces (p.e. r_1) tiene multiplicidad $m > 1$. Entonces la ecuación característica puede ser escrita en la forma:

$$(x - r_1)^m (x - r_2) \dots (x - r_{k-m+1})$$

en cuyo caso la solución de la ecuación en recurrencia viene dada por la expresión:

$$T(n) = \sum_{i=1}^m c_i n^{i-1} r_1^n + \sum_{i=m+1}^k c_i r_{i-m+1}^n$$

donde los coeficientes c_i se determinan a partir de las condiciones iniciales. Veamos un ejemplo en el que la ecuación en recurrencia es:

$$T(n) = 5T(n-1) - 8T(n-2) + 4T(n-3), n \geq 2$$

con las condiciones iniciales $T(k) = k$ para $k = 0, 1, 2$. La ecuación característica que se obtiene es $x^3 - 5x^2 + 8x - 4 = 0$, o lo que es igual $(x-2)^2(x-1) = 0$ y por tanto,

$$T(N) = c_1 2^n + c_2 n 2^n + c_3 1^n$$

De las condiciones iniciales obtenemos $c_1 = 2$, $c_2 = -1/2$ y $c_3 = -2$, por lo que

$$T(N) = 2^{n+1} - n 2^{n-1} - 2 \in \mathcal{O}(n 2^n)$$

Este caso puede ser generalizado de la siguiente forma. Si $r_1, r_2, \dots, r_k$ son las raíces de la ecuación característica de una ecuación en recurrencia homogénea, cada una de multiplicidad m_i , esto es, si la ecuación característica puede expresarse como:

$$(x - r_1)^{m_1} (x - r_2)^{m_2} \dots (x - r_k)^{m_k} = 0$$

entonces la solución a la ecuación en recurrencia viene dada por la expresión:

$$T(n) = \sum_{i=1}^{m_1} c_{1i} n^{i-1} r_1^n + \sum_{i=1}^{m_2} c_{2i} n^{i-1} r_2^n + \dots \sum_{i=1}^{m_k} c_{ki} n^{i-1} r_k^n$$

1.3.4.2 Recurrencias no homogéneas

Consideremos una ecuación de la forma:

$$a_0 T(n) + a_1 T(n-1) + \dots + a_k T(n-k) = b^n p(n)$$

donde los coeficientes a_i y b son números reales, y $p(n)$ es un polinomio en n de grado d . Una primera idea para resolver la ecuación es manipularla para convertirla en homogénea, como muestra el siguiente ejemplo.

Sea la ecuación $T(n) - 2T(n-1) = 3^n$ para $n \geq 2$, con las condiciones iniciales $T(0) = 0$ y $T(1) = 1$. En este caso $b = 3$ y $p(n) = 1$, polinomio en n de grado 0.

Podemos escribir la ecuación de dos formas distintas:

En primer lugar, para $n+1$ tenemos que $T(n+1) - 2T(n) = 3^{n+1}$.

Pero si multiplicamos por 3 la ecuación original obtenemos:

$$3T(n) - 6T(n-1) = 3^{n+1}$$

Restando ambas ecuaciones, conseguimos

$$T(n+1) - 5T(n) + 6T(n-1) = 0,$$

que resulta ser una ecuación homogénea cuya solución, aplicando lo visto anteriormente, es

$$T(n) = 3^n - 2^n \in \Theta(3^n).$$

Estos cambios son, en general, difíciles de ver. Afortunadamente, para este tipo de ecuaciones también existe una fórmula general para resolverlas, buscando sus soluciones entre las funciones que son combinaciones lineales de exponenciales, en donde se demuestra que la ecuación característica es de la forma:

$$(a_0 x^k + a_1 x^{k-1} + a_2 x^{k-2} + \dots + a_k)(x - b)^{d+1} = 0$$

lo que permite resolver el problema de forma similar a los casos anteriores.

Como ejemplo, veamos cómo se resuelve la ecuación en recurrencia que plantea el algoritmo de las torres de Hanoi:

$$T(n) = 2T(n-1) + n.$$

Su ecuación característica es entonces $(x-2)(x-1)^2 = 0$ y, por tanto:

$$T(n) = c_1 2^n + c_2 1^n + c_3 n 1^n \in \Theta(2^n).$$

Generalizando este proceso, supongamos ahora una ecuación de la forma:

$$a_0 T(n) + a_1 T(n-1) + \dots + a_k T(n-k) = b_1^n p_1(n) + b_2^n p_2(n) + \dots + b_s^n p_s(n)$$

donde como en el caso anterior, los coeficientes a_i y b_i son números reales y $p_j(n)$ son polinomios en n de grado d_j . En este caso también existe una forma general de la solución, en donde se demuestra que la ecuación característica es:

$$(a_0 x^k + a_1 x^{k-1} + a_2 x^{k-2} + \dots + a_k)(x - b_1)^{d_1+1} (x - b_2)^{d_2+1} \dots (x - b_s)^{d_s+1} = 0$$

Como ejemplo, supongamos la ecuación

$$T(n) = 2T(n-1) + n + 2^n, n \geq 1,$$

con la condición inicial $T(0) = 1$. En este caso tenemos que $b_1 = 1$, $p_1(n) = n$, $b_2 = 2$ y

$p_2(n) = 1$, por lo que su ecuación característica es $(x-2)^2(x-1)^2 = 0$, lo que da lugar a la expresión final de

$$T(n): T(n) = -2 - n + 2^{n+1} + n 2^n \in \Theta(n 2^n).$$

2 RESUMEN ESQUEMÁTICO

La mejor técnica para diferenciar la eficiencia de los algoritmos es el estudio de los **órdenes de complejidad**. El orden de complejidad se expresa generalmente en términos de la cantidad de datos procesados por el programa, denominada ***n***, que puede ser el tamaño dado o estimado.

En el análisis de algoritmos se considera usualmente el caso peor, si bien a veces conviene analizar igualmente el caso mejor y hacer alguna estimación sobre un caso promedio. Para independizarse de factores coyunturales, tales como el lenguaje de programación, la habilidad del codificador, la máquina de soporte, etc. se suele trabajar con un **cálculo asintótico** que indica cómo se comporta el algoritmo para datos muy grandes y salvo algún coeficiente multiplicativo. Los órdenes de complejidad sólo son importantes para grandes problemas.

El interés principal del análisis de algoritmos radica en saber cómo crece el tiempo de ejecución, cuando el tamaño de la entrada crece. Esto es la *eficiencia asintótica* del algoritmo. Se denomina "asintótica" porque analiza el comportamiento de las funciones en el *límite*, es decir, su tasa de crecimiento.

La notación asintótica se describe por medio de una función cuyo dominio es los números naturales (*N*) estimado a partir de tiempo de ejecución o de espacio de memoria de algoritmos en base a la longitud de la entrada. Se consideran las funciones asintóticamente no negativas.

A un conjunto de funciones que comparten un mismo comportamiento asintótico le denominaremos un orden de complejidad. Habitualmente estos conjuntos se denominan *O*, existiendo una infinidad de ellos; pero nos centraremos en unos pocos de uso frecuente.

Para cada uno de estos conjuntos se suele identificar un miembro *f(n)* que se utiliza como representante de la clase.

conjuntos u órdenes de complejidad	
$O(1)$	orden constante
$O(\log n)$	orden logarítmico
$O(n)$	orden lineal
$O(n \log n)$	
$O(n^2)$	orden cuadrático
$O(n^a)$	orden polinomial ($a > 2$)
$O(a^n)$	orden exponencial ($a > 1$)
$O(n!)$	orden factorial

Tabla 1.14. Órdenes de complejidad

Las funciones de la notación asintótica deben presentarse de la manera más simple posible. Se considerará erróneo presentar un resultado como $O(n^2/2 + n)$, por ejemplo.

Presentar resultados como $E(n)$ o $T(n^2)$ es erróneo. Se debe indicar como $E(n) = O(n)$ o bien $T(n) \in O(n^2)$. (Los símbolos: $=$ o $\in$ son equivalentes en el uso habitual de la notación asintótica).

Los resultados deben expresarse en función del tamaño (o tamaños) de la entrada. Es erróneo el que aparezcan variables locales en la función (por ejemplo, indicar que el orden del problema es $O(i)$ en vez de $O(n)$).

La complejidad de los algoritmos de ordenación más conocidos es:

algoritmo	complejidad
inserción	$O(n^2)$
selección	$O(n^2)$
burbuja	$O(n^2)$
<i>quick sort</i>	$O(n \log(n))$

Tabla 1.15 Órdenes de complejidad de los algoritmos de ordenación

3 GLOSARIO

aleatorizados

Hecho aleatoriamente mediante algún proceso., 20

C ++

C++ es un lenguaje de programación diseñado en 1979 por Bjarne Stroustrup. La intención de su creación fue extender al lenguaje de programación C mecanismos que permiten la manipulación de objetos., 12

combinatoria

La combinatoria es una rama de la matemática perteneciente al área de matemáticas discretas que estudia la enumeración, construcción y existencia de propiedades de configuraciones que satisfacen ciertas condiciones establecidas., 17

componentes de 3GL

En este tipo de lenguajes, los programadores deben indicarle los procedimientos específicos que debe hacer el ordenador para lograr un objetivo. Ejemplos de lenguajes 3GL son C, Fortran, Smalltalk, ADA, C++, C#, Cobol, Delphi, etc., 48

especificación

Es una descripción completa del comportamiento del sistema que se va a desarrollar., 4

firmas,

a firma electrónica es un concepto jurídico, equivalente electrónico al de la firma manuscrita, donde una persona acepta y da por validado el contenido de un mensaje electrónico a través de cualquier medio electrónico que sea legítimo y permitido., 4

Fuzzy Logic

La lógica difusa es una forma de lógica de muchos valores en la que el valor de verdad de las variables puede ser cualquier número real entre 0 y 1. Se emplea para manejar el concepto de verdad parcial, donde el valor de verdad puede variar entre completamente

verdadero y completamente falso., 36

invariante

Un invariante de bucle es una condición [entre las variables del programa] que es necesariamente verdadera inmediatamente antes e inmediatamente después de cada iteración de un bucle., 14

Java

Java es un lenguaje de programación y una plataforma informática que fue comercializada por primera vez en 1995 por Sun Microsystems., 12

Los puntos de historia son un artefacto abstracto que nos permite ponernos de acuerdo más fácilmente en la complejidad, esfuerzo y riesgo que tiene una determinada tarea., 37

mergesort

El algoritmo de ordenamiento por mezcla (merge sort en inglés) es un algoritmo de ordenamiento externo estable basado en la técnica divide y vencerás., 9

metodología ágil

Las metodologías ágiles de desarrollo de software buscan proporcionar en poco tiempo pequeñas piezas de software en funcionamiento para aumentar la satisfacción del cliente., 35

ordenación por inserción

El ordenamiento por inserción (insertion sort en inglés) es una manera muy natural de ordenar para un ser humano, y puede usarse fácilmente para ordenar un mazo de cartas numeradas en forma arbitraria., 9

parámetros por referencia

Un parámetro por referencia tiene por objetivo modificar el contenido de una variable que se le envía a un procedimiento. Si por medio de un parámetro por valor llega un dato a un procedimiento, por un parámetro por referencia se retorna y sale un dato de un procedimiento., 54

- pasan por valor
 - Paso por valor
 - Se crea una copia local de la variable dentro de la función.
 - Paso por referencia
 - Se maneja directamente la variable, los cambios realizados dentro de la función le afectarán también fuera., 54
- Pascal
 - Pascal es un lenguaje de programación creado por el profesor suizo Niklaus Wirth entre los años 1968 y 1969, y publicado en 1970. Su objetivo era crear un lenguaje que facilitara el aprendizaje de programación a sus alumnos., 12
- permutación
 - La palabra "permutación" también se refiere al acto o proceso de cambiar el orden lineal de un conjunto ordenado., 3
- problemas NP-completos
 - Un problema muy complejo se denomina "NP-completo", lo cual básicamente significa que es imposible encontrar un algoritmo eficiente para encontrar una solución óptima., 7
- procedimiento computacional
 - Es el proceso mediante el cual el Sistema de Información toma los datos que requiere para procesar la información. Las entradas pueden ser manuales o automáticas., 3
- programación lineal
 - Es el campo de la programación matemática dedicado a maximizar o minimizar (optimizar) una función lineal, denominada función objetivo, de tal forma que las variables de dicha función estén sujetas a una serie de restricciones expresadas mediante un sistema de ecuaciones o inecuaciones también lineales, 5
 - En ciencias de la computación, y análisis numérico, el pseudocódigo¹ (o lenguaje de descripción algorítmico) es una descripción de alto nivel compacta e informal² del principio operativo de un programa informático u otro algoritmo., 11
- Python
 - Python es un lenguaje de alto nivel de programación interpretado cuya filosofía hace hincapié en la legibilidad de su código, se utiliza para desarrollar aplicaciones de todo tipo, ejemplos Instagram, Netflix, Spotify, Panda 3D, entre otros., 12
- RAM
 - La memoria de acceso aleatorio (Random Access Memory, RAM) es una memoria de almacenamiento a corto plazo. El sistema operativo de ordenadores u otros dispositivos utiliza la memoria RAM para almacenar de forma temporal todos los programas y sus procesos de ejecución., 15
- redes WAN
 - Las WAN son redes a gran escala que abarcan países e incluso continentes. No conectan ordenadores individuales, sino otras redes como LAN o MAN., 10
- transformadas discretas de Fourier
 - La transformada de Fourier, denominada así por Joseph Fourier, es una transformación matemática empleada para transformar señales entre el dominio del tiempo (o espacial) y el dominio de la frecuencia, que tiene muchas aplicaciones en la física y la ingeniería, 17

4 TEST

4.1 PREGUNTAS DE TEST

1. Marque aquellas estrategias de ordenación que tienen un tiempo $O(n)$ cuando se les pide ordenar un vector en el que todos los elementos son iguales:
 - a. ☒ Ordenación por inserción.
 - b. ☒ Ordenación por selección.
 - c. ☐ Ordenación por fusión.
 - d. ☐ Ordenación rápida (pivote primer elemento).
2. ¿Cómo se consigue una medida del tiempo que tarda un algoritmo independiente del procesador?
 - a. ☐ Dividiendo el análisis en casos.
 - b. ☐ Expresándola mediante una función, no un valor.
 - c. ☒ Expresándola en función del número de operaciones elementales.
 - d. ☐ Usando un análisis en tiempo amortizado.
 - e. ☐ No se puede conseguir.
3. El análisis en tiempo promedio.
 - a. ☐ Calcula la media de los tiempos del peor y mejor caso.
 - b. ☒ Calcula la media de los tiempos de las ejecuciones con todas las posibles entradas de un mismo tamaño.
 - c. ☐ Calcula la media de los tiempos de las ejecuciones con las entradas del peor caso de todos los tamaños posibles.
 - d. ☐ Calcula la media de una secuencia de ejecuciones del algoritmo.
4. Marque los conjuntos de cota a los que pertenece la función $n^3/100 + 3n^2$ (no estamos hablando de cotas ajustadas)
 - a. ☐ $O(n^2)$
 - b. ☒ $O(n^3)$
 - c. ☒ $O(n^4)$
 - d. ☒ $O(2n)$
5. Cuando para distintas instancias del problema con el mismo tamaño de entrada, no obtenemos el mismo resultado:
 - a. ☐ No es posible calcular la complejidad a priori y debemos ejecutar el programa varias veces con los mismos datos y obtener el tiempo medio para hallar la complejidad media.
 - b. ☐ Debemos asegurarnos de que los datos son los mismos.
 - c. ☒ Calculamos el máximo y mínimo coste que nos puede dar el algoritmo
 - d. ☐ No se puede aplicar la técnica de paso de programa, ya que esta técnica es para calcular la complejidad a priori.
6. Si $f_1(n) \in O(g_1(n))$ y $f_2(n) \in O(g_2(n))$ entonces:
 - a. ☐ $f_1(n)f_2(n) \in O(\text{máximo } g_1(n), g_2(n))$
 - b. ☒ $f_1(n)f_2(n) \in O(g_1(n) * g_2(n))$
 - c. ☐ Ambas son correctas.
 - d. ☐ Ninguna es correcta.

PREGUNTAS DE TEST

7. Si dos algoritmos tienen la misma complejidad asintótica
- a. ☐ Ninguna de las anteriores
 - b. ☐ Necesitan exactamente el mismo tiempo para su ejecución.
 - c. ☐ Depende del tamaño de datos de entrada.
 - d. ☒ No necesitan exactamente el mismo tiempo para su ejecución.
8. ¿Cuál de los siguientes algoritmos de ordenación tiene menor complejidad?
- a. ☐ Inserción directa
 - b. ☐ Burbuja
 - c. ☒ Mergesort.
9. $\sum_{i=1}^n i^k$ pertenece a:
- a. ☒ $O(n^{k+1})$
 - b. ☐ $O(n^k)$
 - c. ☐ Tanto $O(n^{k+1})$ como $O(n^k)$
 - d. ☐ Ninguna de las anteriores.
10. Si $f(n) \in \Omega(g(n))$ entonces:
- a. ☐ $\exists c, n_0 \in \mathbb{R}^+ : f(n) \geq c * g(n) \forall n$
 - b. ☒ $\exists c, n_0 \in \mathbb{R}^+ : f(n) \geq c * g(n) \forall n \geq 0$
 - c. ☐ $\exists c, n_0 \in \mathbb{R}^+ : f(n) \leq c * g(n) \forall n \geq 0$
 - d. ☐ $\exists c, n_0 \in \mathbb{R}^+ : f(n) < c * g(n) \forall n \geq 0$
11. Dado el polinomio $f(n) = A_m n^m + A_{m-1} n^{m-1} + \dots + A_0$, con $A_0 \in \mathbb{R}^+$, entonces pertenece al orden:
- a. ☐ $\Omega(n^m)$
 - b. ☐ $O(n^m)$
 - c. ☐ $\Theta(n^m)$
 - d. ☒ Las dos primeras respuestas son correctas.

4.2 SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
Pregunta 1	A,B
Pregunta 2	C
Pregunta 3	B
Pregunta 4	B,C,D
Pregunta 5	C
Pregunta 6	B
Pregunta 7	D
Pregunta 8	C
Pregunta 9	A
Pregunta 10	B
Pregunta 11	D

5 BIBLIOGRAFÍA BÁSICA

5.1 BIBLIOGRAFÍA BÁSICA

1. AHO, Alfred H., Hopcroft, John E. y Ullman, Jeffrey D. *Estructuras de Datos y Algoritmos*, Addison-Wesley Iberoamericana, 1988.
2. ALFRED V. AHO, John E. Hopcroft, and Jeffrey D. Ullman. *The Design and Analysis of Computer Algorithms*. Addison-Wesley, 1974.
3. ANANY LEVITIN. *Introduction to the Design & Analysis of Algorithms*. Addison-Wesley, 2007.
4. BAASE, S.; Van Gelder, A. *Computer Algorithms. Introduction to Design and Analysis*. Addison-Wesley, 2000.
5. BALCAZAR, José L. *Apuntes sobre el Cálculo de la Eficiencia de los Algoritmos*, Universidad Politécnica de Cataluña, España.
6. BRASSARD, G., Bratley, P., *Fundamentals of Algorithmics*. Prentice Hall, 1995.
7. CORMEN Thomas, Leiserson Charles and Rivest Ronald. *Introduction to Algorithms*. The MIT Press, 1990.
8. DEXTER C. KOZEN. *The Design and Analysis of Algorithms*. Springer, 1992.
9. DONALD E. KNUTH. *Fundamental Algorithms*, volume 1 of *The Art of Computer Programming*. Addison-Wesley, 1968. Third edition, 1997.
10. ELLIS HOROWITZ, Sartaj Sahni, and Sanguthevar Rajasekaran. *Computer Algorithms*. Computer Science Press, 1998.
11. G. H. GONNET. *Handbook of Algorithms and Data Structures*. Addison-Wesley, 1984.
12. GILLES BRASSARD and Paul Bratley. *Fundamentals of Algorithmics*. Prentice Hall, 1996.82
13. GIMÉNEZ Cánovas, Domingo. *Apuntes y Problemas de Algorítmica*. Facultad de Informática. Universidad de Murcia, 2001.
14. GONNET, G. H., Baeza-Yates, R. *Handbook of Algorithms and Data Structures*, Addison-Wesley, 1991.
15. HERBERT S. WILF. *Algorithms and Complexity*. A K Peters, second edition, 2002.
16. HOROWITZ, E.; Sahni, S. *Fundamentals of Data Structures*. Computer Science Press, 1976.
17. JON L. BENTLEY. *Writing Efficient Programs*. Prentice Hall, 1982.
18. MICHAEL T. GOODRICH AND ROBERTO TAMASSIA. *Algorithm Design: Foundations, Analysis, and Internet Examples*. John Wiley & Sons, 2001.
19. JON KLEINBERG AND EVA TARDOS. *Algorithm Design*. Addison-Wesley, 2006.
20. JOAO SETUBAL AND JOAO MEIDANIS. *Introduction to Computational Molecular Biology*. PWS Publishing Company, 1997.
21. PUBLISHING COMPANY, 1997.
22. JEFFREY H. KINGSTON. *Algorithms and Data Structures: Design, Correctness, Analysis*. Addison-Wesley, second edition, 1997.
23. KURT MEHLHORN. *Sorting and Searching*, volume 1 of *Data Structures and Algorithms*. Springer, 1984.
24. KURT MEHLHORN. *Graph Algorithms and NP-Completeness*, volume 2 of *Data Structures and Algorithms*. Springer, 1984.
25. KURT MEHLHORN. *Multidimensional Searching and Computational Geometry*, volume 3 of *Data Structures and Algorithms*. Springer, 1984.
26. KNUTH, Donald E. *"Fundamental algorithms"*, volume 1 de *"The Art of Computer Programming"*, segunda edición, Addison-Wesley Publishing Company, Reading, Massachusetts, 1973.

27. KNUTH, Donald E. "Sorting and Searching", volume 3 de "The Art of Computer Programming", segunda edición, Addison-Wesley Publishing Company, Reading, Massachusetts, 1973.
28. MICHA HOFRI. Analysis of Algorithms. Oxford University Press, 1995
29. PAUL W. PURDOM, Jr. and Cynthia A. Brown. The Analysis of Algorithms. Holt, Rinehart and Winston, 1985.
30. PARBERRY Ian. *Problems on Algorithms*. Prentice Hall, 1995.
31. PAVEL A. PEVZNER. Computational Molecular Biology: An Algorithmic Approach. The MIT Press, 2000.
32. SANJOY DASGUPTA, Christos Papadimitriou, and Umesh Vazirani. Algorithms. McGraw-Hill, 2008.
33. RICHARD JOHNSONBAUGH and Marcus Schaefer. Algorithms. Pearson Prentice Hall, 2004.
34. ROBERT SEDGEWICK and PHILIPPE FLAJOLET. An Introduction to the Analysis of Algorithms. Addison-Wesley, 1996.
35. ROBERT SEDGEWICK, Kevin Wayne Algorithms - 4th ed. Addison-Wesley.
36. SARA BAASE and ALAN VAN GELDER. Computer Algorithms: Introduction to Design and Analysis. Addison-Wesley, third edition, 2000
37. STEVEN S. SKIENA. The Algorithm Design Manual. Springer, second edition, 1998.
38. SYMONS CR Software Sizing and Estimating MKII FPA. John Wiley and Sons, 1991.
39. UDI MANBER. Introduction to Algorithms: A Creative Approach. Addison-Wesley, 1989
40. WEISS, M. A., *Estructuras de datos y algoritmos*. Addison-Wesley Iberoamericana, 1995.
41. WIRTH, Niklaus. *Algoritmos y Estructuras de Datos*. Pentice Hall, 1987

5.2 BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

- 1 ALBRECHT, ALLAN J. 1979. Measuring Application Development Productivity. Proc Of IBM applications. Development Joint SHARE/GUIDE Symposium, Monterrey, Páginas 83-92.
- 2 DAN GUSFIELD. Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology. Cambridge University Press, 1997.
- 3 DONALD E. KNUTH. Sorting and Searching, volume 3 of The Art of Computer Programming. Addison-Wesley, 1973. Second edition, 1998.
- 4 DONALD E. KNUTH. Seminumerical Algorithms, volume 2 of The Art of Computer Programming. Addison-Wesley, 1969. Third edition, 1997.
- 5 EDWARD M. REINGOLD, Jürg Nievergelt, and Narsingh Deo. Combinatorial Algorithms: Theory and Practice. Prentice Hall, 1977
- 6 JON L. BENTLEY. Programming Pearls. Addison-Wesley, 1986.
- 7 KARNER, G. Resource estimation for objectory projects in *Objective systems* (1993). SF AB.
- 8 MICHAEL S. WATERMAN. Introduction to Computational Biology, Maps, Sequences and Genomes. Chapman & Hall, 1995.

¹ **Notas del capítulo:** Hay muchos textos excelentes sobre el tema general de los algoritmos, Halstead, 1977 incluidos los de Aho, Hopcroft y Ullman [5, 6]; Baase y Van Gelder [28]; Brassard y Bratley [54]; Dasgupta, Papadimitriou y Vazirani [82]; Goodrich y Tamassia [148]; ofri [175]; Horowitz, Sahni y Rajasekaran [181]; Johnsonbaugh y Schaefer [193]; Kingston [205]; Kleinberg y Tardos [208]; Knuth [209, 210, 211]; Kozen [220]; Levitina [235]; Manber [242]; Mehlhorn [249, 250, 251]; Purdom y Brown [287]; Reingold, Nievergelt y Deo [293]; Sedgewick [306]; Sedgewick y Flajolet [307]; Skiena [318]; y Wilf [356]. Bentley [42, 43] y Gonnet [145]; Gusfield [156], Pevzner [275], Setubal y Meidanis [310]. Se

BIBLIOGRAFÍA

pueden encontrar descripciones generales de los algoritmos utilizados en biología computacional en los libros de texto de Gusfield [156], Pevzner [275], Setubal y Meidanis [310] y Waterman [350].

"En 1968, Knuth publicó el primero de tres volúmenes con el título general "El arte de la programación informática" [209, 210, 211]. El primer volumen marcó el comienzo del estudio moderno de algoritmos informáticos con un enfoque en el análisis del tiempo de ejecución, y la serie completa sigue siendo una referencia interesante y valiosa para muchos de los temas presentados aquí. Según Knuth, la palabra "algoritmo" se deriva del nombre "al-Khowârizmî", un matemático persa del siglo IX. Aho, Hopcroft y Ullman [5] abogaron por el análisis asintótico de algoritmos, utilizando las notaciones que introduce el Capítulo 3, incluida la θ -notación, como un medio para comparar el rendimiento relativo. También popularizaron el uso de relaciones de recurrencia para describir los tiempos de ejecución de los algoritmos recursivos. Knuth [211] proporciona un tratamiento enciclopédico de muchos algoritmos de clasificación. Su comparación de algoritmos de clasificación incluye análisis de conteo de pasos exactos, como el que realizamos aquí para la "clasificación por inserción". La discusión de Knuth sobre la ordenación por inserción abarca varias variaciones del algoritmo. El más importante de ellos es el ordenamiento de "Shell", introducido por D. L. Shell, que usa ordenamiento por inserción en subsecuencias periódicas de la entrada para producir un algoritmo de ordenamiento más rápido. Knuth también describe la ordenación por combinación (mergesort). Menciona que en 1938 se inventó un clasificador mecánico capaz de mezclar dos barajas de cartas perforadas en una sola pasada. J. von Neumann, uno de los pioneros de la informática, aparentemente escribió un programa para la clasificación por combinación en el ordenador EDVAC en 1945. Gries [153] describe la historia temprana de demostrar que los programas son correctos, y atribuye a P. Naur el primer artículo de este campo. Gries atribuye invariantes de bucle a R. W. Floyd. El libro de texto de Mitchell [256] describe un progreso más reciente en la demostración de que los programas son correctos.